UNIVERSIDADE CATÓLICA DE SANTOS

Ciência da Computação

UV MAPPING DINÂMICO E PROCEDURAL

Computação Gráfica e Processamento de Imagens

Grupo: MIA

Integrantes:

Arthur Almeida Lima - 3093901
Felipe Barbosa dos Santos - 4324223
João Gabriel Henriques Cardoso - 6100450
Lucas Carmona Neto - 8342055
Lucas Cerqueira Galvão - 4477944

Santos – SP
2026

Sumário

1 Introdução **4**

2 Fundamentação teórica **5**

2.1 UV Mapping tradicional 5

2.2 UV Mapping dinâmico 5

2.3 Mapeamento procedural 6

2.4 Quando usar cada abordagem 6

2.5 Limitações 7

2.6 Custo computacional 9

3 Equações e funcionamento matemático **11**

3.1 Coordenadas UV do vértice 11

3.2 Transformação da posição 11

3.3 Interpolação baricêntrica com correção de perspectiva 12

3.4 Translação dependente do tempo 12

3.5 Ondulação (deformação não linear) 13

3.6 Repetição da textura (endereçamento REPEAT) 13

3.7 Escala 13

3.8 Rotação 14

3.9 Transformação combinada em coordenadas homogêneas 14

3.10 Função procedural 14

3.11 Seleção do nível de mipmap 15

3.12 Iluminação difusa 16

3.13 Resumo: equação, estágio e frequência 16

4 Relação com o Vulkan e arquitetura do pipeline **18**

4.1 Visão geral 18

4.2 Recursos usados pela técnica 20

4.3 Fluxo de dados entre os estágios 21
4.4 Descriptor Sets e Bindings 21
4.5 Push constants 23
4.6 Render Pass e Subpass 23
4.7 Sincronização: barreiras, layouts e dependências 24
4.8 Onde cada cálculo é feito 25
4.9 Otimizações: aplicadas e pendentes 27
4.10 Justificativa das decisões arquiteturais 28

5 Shaders: pseudocódigo e implementação 29

5.1 Objetivo de cada shader 29
5.2 Fluxo entre os shaders 29
5.3 Pseudocódigo completo 30
5.4 Entradas, saídas e recursos 40
5.5 Equações e variáveis 40
5.6 Condições e laços 41
5.7 Amostragem da textura 42
5.8 Tratamento de limites e valores inválidos 42
5.9 Resultado esperado 43
5.10 Relação entre o pseudocódigo e a técnica 44

6 Aplicações práticas 45

6.1 Motores de jogo: UV dinâmico como recurso de material 45
6.2 Ferramentas de autoria de texturas procedurais 45
6.3 Modelagem procedural de mundos virtuais 46
6.4 Resumo 47

7 Análise em jogos 48

7.1 Rocket League 49
7.2 Marvel's Spider-Man Remastered 52
7.3 Demonstração na implementação do grupo 55

7.4 Comparação 55

8 Resumo das publicações utilizadas **57**

8.1 Heckbert (1986) 57

8.2 Dong et al. (2020) 57

8.3 Smelik et al. (2014) 58

8.4 Relação entre as publicações 59

9 Conclusão **60**

Referências **62**

A Checklist dos requisitos **64**

1 Introdução

O mapeamento de texturas (*texture mapping*) é uma técnica fundamental da Computação Gráfica para associar informações bidimensionais à superfície de objetos tridimensionais. Na sua forma tradicional, as coordenadas U e V estabelecem a correspondência entre pontos de uma superfície 3D e posições em uma textura 2D. Heckbert (1, p. 1) apresenta a técnica como uma forma de aumentar a riqueza visual das imagens "com um aumento relativamente pequeno de computação", sem exigir que todos os detalhes sejam modelados geometricamente.

A partir dessa base, o mapeamento pode ser modificado durante a execução. No *UV mapping dinâmico*, as coordenadas sofrem translação, escala, rotação ou deformações que dependem do tempo, de modo que a textura se move sobre uma malha parada. No *mapeamento procedural*, a cor ou a própria coordenada é calculada por uma função matemática, em vez de lida de uma imagem (2, p. 1).

Este trabalho apresenta uma revisão bibliográfica da técnica, apoiada em três levantamentos: o trabalho clássico de Heckbert sobre texture mapping (1), o levantamento de Dong et al. sobre geração procedural de texturas bidimensionais (2) e o de Smelik et al. sobre modelagem procedural de mundos virtuais (3). A revisão é acompanhada de uma implementação em Vulkan (pasta `código/`), que renderiza um modelo glTF em cinco modos: textura original, UV dinâmico, padrão procedural, combinação dos dois e grade de depuração do UV. A implementação é usada ao longo do texto como exemplo concreto de cada equação, de cada recurso do Vulkan e de cada limitação discutida.

O documento está organizado da seguinte forma:

- o Capítulo 2 apresenta os fundamentos e discute quando usar cada abordagem, suas limitações e seu custo computacional;
- o Capítulo 3 apresenta as equações na ordem em que são executadas no pipeline, indicando o estágio e a frequência de cada uma;
- o Capítulo 4 trata da relação com o Vulkan e da arquitetura do pipeline: diagrama, recursos, descriptor sets, push constants, render pass, sincronização e otimizações;
- o Capítulo 5 apresenta os shaders comentados, do pseudocódigo à implementação;
- o Capítulo 6 analisa sete produtos reais que usam a técnica;
- o Capítulo 7 analisa *Rocket League* e *Marvel's Spider-Man Remastered*, com capturas próprias marcadas, vídeos do grupo e uma demonstração na implementação;
- o Capítulo 8 resume as três publicações, e o Capítulo 9 conclui o trabalho.

2 Fundamentação teórica

2.1 UV Mapping tradicional

O UV mapping estabelece uma correspondência entre a superfície de um objeto tridimensional e uma imagem bidimensional. As coordenadas U e V normalmente são normalizadas no intervalo $[0, 1]$. Dessa maneira, um vértice do modelo carrega, além de sua posição e de sua normal, uma coordenada que indica qual ponto da textura corresponde àquele ponto da superfície.

Heckbert (1, p. 2) descreve o processo como a composição de dois mapeamentos. O primeiro é a *parametrização da superfície*, que leva o espaço de textura (u, v) ao espaço do objeto. O segundo são as transformações de modelagem e de visualização, que levam o objeto ao espaço de tela (x, y). Na prática, o espaço 3D intermediário costuma ser "esquecido": o que interessa ao renderizador é a relação direta entre cada pixel e um ponto da textura. Em superfícies paramétricas essa parametrização é natural, mas em malhas poligonais ela precisa ser definida explicitamente (1, p. 3). É exatamente isso que o artista faz ao "desdobrar" um modelo e salvar o atributo `TEXCOORD_0` em um arquivo glTF, como o modelo `Duck.glb` usado na implementação deste trabalho.

Entre as ordens de varredura analisadas por Heckbert, a mais comum é a varredura em ordem de tela (*inverse mapping*): para cada pixel, encontra-se a sua pré-imagem no espaço de textura e filtra-se essa região (1, p. 3). Essa é a lógica do fragment shader em uma GPU moderna: o rasterizador gera um fragmento por pixel coberto, interpola o UV dos vértices e o shader consulta a textura naquele ponto por meio de um sampler.

Heckbert também separa o texture mapping em duas partes: o mapeamento geométrico, que deforma a textura sobre a superfície, e a filtragem, necessária para evitar aliasing (1, p. 1). Ambas reaparecem quando o mapeamento passa a ser dinâmico ou procedural.

2.2 UV Mapping dinâmico

O UV mapping dinâmico consiste em modificar as coordenadas de textura durante a execução, sem alterar a malha. Uma transformação simples é a translação:

$$u' = u + \Delta u, \qquad v' = v + \Delta v.$$

Se Δu ou Δv variarem com o tempo, a textura desliza sobre a superfície. Em termos da composição de Heckbert, o que muda a cada quadro é a parametrização da superfície: a geometria e a câmera continuam as mesmas, mas cada ponto da superfície passa a apontar

para outro ponto da textura. Também podem ser aplicadas escala, rotação e deformações não lineares (ondulações), cujas equações e o momento em que cada uma é executada no pipeline são detalhados no Capítulo 3.

Como as coordenadas transformadas saem do intervalo $[0, 1]$, a técnica depende de um modo de endereçamento que repita a textura. No Vulkan, isso é feito pelo sampler com `VK_SAMPLER_ADDRESS_MODE_REPEAT`, que é o modo usado na implementação deste trabalho.

2.3 Mapeamento procedural

No mapeamento procedural, a cor (ou a própria coordenada) é produzida por uma função, em vez de lida de uma imagem. Dong et al. definem a geração procedural de texturas como "o processo de criar texturas usando modelos matemáticos", cuja entrada pode ser um conjunto de parâmetros, valores aleatórios de funções de ruído ou imagens existentes (2, p. 1). Heckbert já previa essa possibilidade ao afirmar que uma textura pode ser representada "por um array ou por uma função matemática", citando texturas de nuvens, madeira e mármore (1, p. 1), e as *solid textures*, que dispensam a parametrização porque são avaliadas diretamente na posição 3D (1, p. 3).

Uma formulação simplificada é:

$$C = f(P, t),$$

em que P representa uma posição (no espaço UV ou no espaço 3D), t uma variável temporal e f a função procedural.

Dong et al. organizam os métodos em funções *estruturadas* (com aparência de regularidade, como autômatos celulares, reação-difusão, posicionamento de textons, diagramas de Voronoi e padrões islâmicos) e *não estruturadas* (com aparência aleatória, em que predominam as funções de ruído, divididas em métodos no domínio da frequência e no domínio espacial) (2, pp. 3–4). O padrão procedural implementado neste trabalho (modo 3) é uma soma de três senoides avaliadas sobre coordenadas distorcidas. A ideia de compor o padrão com senoides vem das séries de Fourier usadas como função de textura, que Heckbert registra (1, p. 10). A distorção aplicada às coordenadas, porém, faz com que o resultado não seja estritamente uma série de Fourier.

2.4 Quando usar cada abordagem

As três abordagens não competem entre si. Cada uma resolve um problema diferente, e a escolha depende do conteúdo da superfície. Dong et al. chegam à mesma conclusão para

os métodos procedurais: o mais adequado "depende dos requisitos da superfície desejada" (2, p. 22). A Tabela 2.1 resume os critérios usados neste trabalho.

Tabela 2.1: Critérios de escolha entre UV tradicional, UV dinâmico e procedural.

Situação	**Abordagem**	**Motivo**
Detalhe único pintado pelo artista (rosto, logotipo, decalque, texto)	UV tradicional	A informação é posicional: deslocá-la ou gerá-la por função destruiria o conteúdo. Funções procedurais são difíceis de controlar para reproduzir um desenho específico (2, p. 2).
Movimento contínuo e repetitivo (água, lava, esteira, tela, campo de energia)	UV dinâmico	Uma única textura repetível gera o movimento inteiro. O custo é uma amostragem e poucas operações por fragmento.
Grandes áreas sem repetição visível (terreno, pedra, nuvem)	Procedural	Não há limite de tamanho ou resolução, nem costuras ou repetições indesejadas (2, p. 2).
Muitas variações de um mesmo material	Procedural	Variar os parâmetros gera uma família de texturas relacionadas, em vez de uma única imagem fixa (2, p. 2).
Conteúdo em escala de mundo (cidades, vegetação)	Modelagem procedural, em geral *offline*	Amplificação de dados: poucos parâmetros produzem grande variedade de modelos (3, p. 1).
Reproduzir uma textura real específica	Textura fotografada ou pintada	Ajustar um modelo procedural a um exemplo dado é apontado como um desafio ainda difícil (2, p. 23).

Na prática, as técnicas são combinadas. O modo 4 da implementação, por exemplo, aplica UV dinâmico sobre a textura original do modelo e sobrepõe ondas de um padrão procedural apenas nas regiões de maior "energia". O UV dinâmico também não é indicado para texturas que não sejam repetíveis (*tileable*): ao deslizar uma imagem cujas bordas não se encaixam, a linha de emenda aparece atravessando a superfície.

2.5 Limitações

Distorção e costuras herdadas da parametrização. UV dinâmico e procedural em espaço UV não corrigem a parametrização original: apenas a reutilizam. Se o artista esticou a textura em uma região, o padrão animado também fica esticado ali. Nas costuras

do desdobramento (*seams*), dois pontos vizinhos na superfície correspondem a pontos distantes na textura, e um deslocamento contínuo fica descontínuo na emenda. Heckbert observa o mesmo problema no bump mapping, cujas tangentes nas direções u e v precisam ser contínuas através das costuras dos polígonos (1, p. 11). O modo 5 da implementação (grade UV) existe justamente para tornar visíveis o esticamento e as costuras do modelo.

Aliasing. Uma superfície vista obliquamente pode criar frequências arbitrariamente altas na tela (1, p. 6). Em uma textura armazenada, isso é contornado com pré-filtragem (mipmaps). Uma função procedural, porém, não tem mipmaps: ela é avaliada no centro de cada fragmento, e qualquer frequência acima do limite do pixel vira ruído ou padrão de moiré. Heckbert indica que funções de textura arbitrárias são mais difíceis de filtrar que arrays e exigem amostragem estocástica adaptativa (1, p. 11). Dong et al. apontam que o próprio ruído de Perlin original "é propenso a problemas de aliasing e perda de detalhe" (2, p. 13). Na implementação deste trabalho, as linhas da grade procedural usam `smoothstep` com largura fixa no espaço UV e, por isso, apresentam serrilhado quando o modelo fica pequeno na tela.

Controle artístico. Segundo Dong et al., "é difícil prever a aparência da textura ao escolher um conjunto de parâmetros", mesmo para profissionais experientes (2, p. 2). Em métodos como a reação-difusão, pequenas variações nos parâmetros podem gerar texturas completamente diferentes (2, p. 5). Smelik et al. apontam a baixa controlabilidade como o principal motivo de os métodos procedurais ainda não substituírem a modelagem manual (3, p. 1): uma pequena alteração em uma regra provoca uma reação em cadeia no modelo inteiro, e o uso se reduz a tentativa e erro (3, p. 12).

Precisão numérica do tempo. O parâmetro t é um `float` de 32 bits que cresce sem limite. A partir de $t = 2^{16} \approx 65.500\,\text{s}$ (cerca de 18 horas de execução), o menor incremento representável de t passa a ser $2^{-7} \approx 0{,}008\,\text{s}$, metade da duração de um quadro a 60 Hz, e a animação começa a "engasgar". As coordenadas deslocadas, como $u + 0{,}10\,t$, crescem junto com t e perdem precisão da mesma forma. Uma correção é a CPU calcular o tempo em precisão dupla e enviá-lo já "dando a volta" ($t \bmod T$). Para que essa volta não cause um salto, todos os movimentos precisam se repetir em T: cada velocidade e cada frequência tem de completar um número inteiro de ciclos nesse intervalo. Com as constantes atuais isso não acontece (a translação se repete a cada 10 s, e o termo $\sin(\ldots + 2t)$ a cada π s), então seria preciso ajustar as constantes ou aceitar um pequeno salto a cada T, com T grande. A implementação atual não faz nenhuma das duas coisas.

Aparência, não forma. O UV dinâmico muda o que se vê na superfície, mas não muda a silhueta. Ondas em uma superfície de água feitas apenas com UV não fazem o contorno do objeto ondular. Para isso, é preciso deslocar vértices (*displacement*).

2.6 Custo computacional

Heckbert destaca que o texture mapping aumenta a riqueza visual "com um aumento relativamente pequeno de computação" (1, p. 1). Esse custo, porém, depende de onde cada operação é executada e de quantas vezes ela se repete. A diferença de escala é grande: o modelo usado na implementação tem 2.399 vértices, enquanto uma janela de 800×600 tem 480.000 pixels, e uma tela Full HD (1920×1080) tem 2.073.600. Uma operação no fragment shader pode, portanto, executar centenas de vezes mais que a mesma operação no vertex shader.

A Tabela 2.2 conta as operações por fragmento de cada modo da implementação, a partir do código de `uv_mapping.frag`.

Tabela 2.2: Custo por fragmento dos modos implementados (contagem a partir do shader).

Modo	**Amostras**	**sin/cos**	**Memória de textura**
1 – Textura original	1	0	1 MiB (512×512 RGBA8)
2 – UV dinâmico	1	2	1 MiB (a mesma textura)
3 – Procedural	0	6	0 (só o código do shader)
4 – Textura + procedural	1	8	1 MiB
5 – Grade UV	0	0	0

No pior caso, com o modelo cobrindo uma tela Full HD a 60 quadros por segundo, o modo 3 avalia cerca de $2{,}07 \times 10^6 \times 6 \times 60 \approx 7{,}5 \times 10^8$ funções trigonométricas por segundo. Uma GPU atual processa esse volume com folga, mas ele cresce linearmente com a resolução (quatro vezes mais em 4K), com a sobreposição de fragmentos (*overdraw*) e com o número de camadas procedurais.

O procedural troca memória por cálculo. Uma textura de 4096×4096 em RGBA8 ocupa 64 MiB, e com a cadeia completa de mipmaps chega a 4/3 disso, cerca de 85 MiB (1, p. 9). O padrão procedural equivalente não ocupa memória de textura, mas paga o custo em instruções a cada fragmento e a cada quadro. Na filtragem, Heckbert mostra que a interpolação trilinear sobre uma pirâmide de mipmaps custa 8 acessos e 7 multiplicações por pixel, com custo constante, enquanto a convolução direta tem custo proporcional ao número de texels acessados, isto é, à área filtrada (1, pp. 8–9, 11). O autor também observa que a pré-filtragem só compensa quando a textura é reutilizada em vários quadros (1, p. 10). Essa condição vale para o UV dinâmico, que reaproveita a mesma imagem filtrada, mas não para um padrão recalculado a cada quadro.

Sobre a viabilidade em tempo real, a literatura consultada é cautelosa. Dong et al. geraram as amostras do estudo em 512×512 (2, p. 19) e relatam que todos os métodos levaram, em geral, menos de um segundo por imagem em um computador de mesa, motivo

pelo qual não apresentam medidas de complexidade (2, p. 21). Essa medida é de geração de uma imagem completa, e não de avaliação por fragmento em tempo real. Smelik et al. afirmam que métodos procedurais "frequentemente não são rápidos o suficiente" para retorno interativo (3, p. 14), com exceção de algoritmos como o fBm, em que cada ponto é calculado de forma independente e paralelizável na GPU (3, p. 3). O padrão do modo 3 tem essa mesma característica: cada fragmento é independente dos vizinhos, o que permite executá-lo inteiro no fragment shader.

Uma regra prática decorre dessas observações. Operações lineares no UV (translação, escala, rotação) produzem o mesmo resultado se forem aplicadas nos vértices e depois interpoladas, e por isso podem ir para o vertex shader. Já operações não lineares (ondulações com sin, funções de ruído) precisam ficar no fragment shader, porque, avaliadas apenas nos vértices, ficariam dependentes da densidade da malha.

3 Equações e funcionamento matemático

Este capítulo apresenta as equações da técnica na ordem em que são executadas durante a renderização de um quadro. Para cada uma, é indicado o estágio do pipeline que a executa, quantas vezes ela é avaliada e onde aparece na implementação (`código/shaders/uv_mapping.vert`, `uv_mapping.frag` e os arquivos de `código/src/`). A Tabela 3.1, ao final, resume essa correspondência.

3.1 Coordenadas UV do vértice

Onde: atributo de vértice, lido pelo estágio *Vertex Input*. **Quando:** uma vez por vértice, a cada quadro.

$$(u, v), \qquad 0 \leq u \leq 1, \quad 0 \leq v \leq 1. \tag{3.1}$$

u representa a posição horizontal na textura e v a vertical. Por serem normalizadas, as coordenadas não dependem da resolução da imagem: o mesmo modelo funciona com uma textura de 512×512 ou de 4096×4096. Na implementação, cada vértice ocupa 32 bytes (posição, normal e UV, todos em `float`), e o UV vem do atributo `TEXCOORD_0` do glTF, que chega ao shader como `layout(location = 2) in vec2 inUV`.

3.2 Transformação da posição

Onde: vertex shader. **Quando:** uma vez por vértice.

$$p_{\text{clip}} = P \cdot V \cdot M \cdot p_{\text{objeto}}. \tag{3.2}$$

Esta é a segunda metade da composição descrita por Heckbert (objeto → tela) (1, p. 2). As três matrizes são calculadas uma vez por quadro na CPU (`updateUniformBuffer`) e lidas do Uniform Buffer. O UV não é alterado aqui: o vertex shader da implementação apenas o repassa (`fragUV = inUV`), porque as transformações não lineares precisam ser feitas por fragmento (ver Seção 3.5).

O resultado p_{clip} ainda não é a posição na tela. Depois do vertex shader, o hardware divide x, y e z por w (divisão perspectiva) e aplica o viewport. O valor z/w, que na implementação fica entre 0 e 1 para os pontos visíveis, é a profundidade usada no teste de profundidade.

3.3 Interpolação baricêntrica com correção de perspectiva

Onde: rasterizador (estágio fixo, não programável). **Quando:** uma vez por fragmento.

Para um ponto dentro de um triângulo, com coordenadas baricêntricas $\alpha + \beta + \gamma = 1$, a interpolação linear seria:

$$UV = \alpha\, UV_0 + \beta\, UV_1 + \gamma\, UV_2. \tag{3.3}$$

Heckbert mostra, porém, que interpolar u e v linearmente no espaço de tela "nunca dá o efeito correto do encurtamento perspectivo" e que o erro é evidente em animação. A forma correta exige uma divisão por pixel, $u = (ax + by + c)/(gx + hy + i)$ (1, p. 5). As GPUs implementam uma forma equivalente, interpolando UV/w e $1/w$ e dividindo um pelo outro, em que w é a coordenada homogênea de cada vértice após a Equação (3.2):

$$UV = \frac{\alpha \dfrac{UV_0}{w_0} + \beta \dfrac{UV_1}{w_1} + \gamma \dfrac{UV_2}{w_2}}{\dfrac{\alpha}{w_0} + \dfrac{\beta}{w_1} + \dfrac{\gamma}{w_2}}. \tag{3.4}$$

No Vulkan, esse é o comportamento padrão de toda variável `out` do vertex shader, a menos que ela seja declarada `noperspective`. É por isso que `fragUV` chega ao fragment shader já correto, sem nenhum código adicional.

3.4 Translação dependente do tempo

Onde: fragment shader, função `dynamicUV`. **Quando:** uma vez por fragmento, nos modos 2 e 4.

$$u' = u + s_u\, t, \qquad v' = v + s_v\, t. \tag{3.5}$$

s_u e s_v são velocidades em "texturas por segundo" e t é o tempo em segundos. Na implementação, $s_u = 0{,}10$ e $s_v = 0$ (`uv.x += time * 0.10`): a textura atravessa a superfície inteira a cada 10 segundos. O valor de t é medido na CPU com `std::chrono` e enviado no campo `time` do Uniform Buffer. É a única entrada que muda de um quadro para outro. Por ser linear, essa operação poderia ser feita no vertex shader com o mesmo resultado, já que a Equação (3.4) preserva combinações lineares.

3.5 Ondulação (deformação não linear)

Onde: fragment shader, função `dynamicUV`. **Quando:** uma vez por fragmento, nos modos 2 e 4.

$$u'' = u' + A\sin(k\,v + \omega_u t), \qquad v'' = v + A\cos(k\,u'' + \omega_v t). \tag{3.6}$$

Com $A = 0{,}015$, $k = 12$, $\omega_u = 2$ e $\omega_v = 1{,}5$, cada região da superfície recebe um deslocamento diferente, o que dá a impressão de líquido. Diferentemente da translação, esta equação não pode ir para o vertex shader: avaliada só nos vértices e depois interpolada, a senoide seria aproximada por segmentos de reta entre vértices, e o resultado dependeria da densidade da malha.

3.6 Repetição da textura (endereçamento REPEAT)

Onde: unidade de amostragem (sampler), em hardware. **Quando:** a cada leitura de textura.

$$u_{\text{final}} = u'' - \lfloor u'' \rfloor, \qquad v_{\text{final}} = v'' - \lfloor v'' \rfloor. \tag{3.7}$$

Como $u' = u + 0{,}10\,t$ cresce sem limite, a coordenada precisa "dar a volta" para continuar dentro da textura. Isso não é feito no shader: é configurado no sampler com `VK_SAMPLER_ADDRESS_MODE_REPEAT` (`createTextureSampler`, em `código/src/VulkanTexture.cpp`). Sem esse modo, a leitura ficaria presa à borda da textura assim que u' passasse de 1.

3.7 Escala

Onde: fragment shader. **Quando:** uma vez por fragmento, nos modos 3, 4 e 5.

$$u' = k\,u, \qquad v' = k\,v. \tag{3.8}$$

O fator de escala k é adimensional: valores $k > 1$ repetem a textura ou o padrão k vezes ao longo da superfície. Combinada com a Equação (3.7) (ou com a função `fract` do GLSL), a escala gera ladrilhamento. Na implementação, ela aparece em `fract(uv * 8.0)`, que cria a grade 8×8 do padrão procedural, e em `uv * 10.0`, que cria a grade de depuração 10×10 do modo 5.

3.8 Rotação

Onde: fragment shader ou vertex shader (por ser linear). **Quando:** uma vez por fragmento ou por vértice.

$$\begin{pmatrix} u' \\ v' \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} u - c_u \\ v - c_v \end{pmatrix} + \begin{pmatrix} c_u \\ c_v \end{pmatrix}. \tag{3.9}$$

O ponto (c_u, c_v) é o centro da rotação, normalmente (0,5; 0,5), para girar a textura em torno do seu centro e não do canto. A implementação não usa rotação. Ela entraria em efeitos como um redemoinho ou um ventilador. Se θ for igual para toda a superfície, $\cos\theta$ e $\sin\theta$ devem ser calculados uma única vez na CPU e enviados prontos, evitando duas funções trigonométricas por fragmento.

3.9 Transformação combinada em coordenadas homogêneas

Onde: montada na CPU; aplicada no vertex shader ou no fragment shader. **Quando:** montada uma vez por quadro.

$$\begin{pmatrix} u' \\ v' \\ 1 \end{pmatrix} = \underbrace{T(\mathbf{s}\,t)\; R(\theta)\; S(k)}_{M_{UV}} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix}. \tag{3.10}$$

Usando uma matriz 3×3, translação, rotação e escala viram um único produto matriz-vetor. Esta é a forma mais econômica de aplicar várias transformações lineares: a CPU multiplica as três matrizes uma vez por quadro e o shader faz uma única multiplicação, independentemente de quantas operações foram combinadas.

3.10 Função procedural

Onde: fragment shader, função `proceduralPattern`. **Quando:** uma vez por fragmento, nos modos 3 e 4.

A forma geral é $C = f(UV, t)$. Na implementação, o UV é primeiro deslocado e distorcido, com $u \mathrel{+}= 0{,}40\,t$, $v \mathrel{+}= 0{,}10\sin(1{,}2\,t)$ e uma ondulação de amplitude 0,08 parecida com a da Equação (3.6), com uma diferença: os dois deslocamentos são calculados a partir do mesmo UV e só depois somados. Sobre esse UV, f é uma soma de três ondas planas:

$$e = \frac{1}{2} + \frac{1}{6}\sum_{i=1}^{3} \sin(\mathbf{k}_i \cdot UV + \omega_i t)\,, \qquad \begin{aligned} \mathbf{k}_1 &= (20, 8), \quad \omega_1 = 4, \\ \mathbf{k}_2 &= (-6, 24), \quad \omega_2 = -3, \\ \mathbf{k}_3 &= (18, 18), \quad \omega_3 = 2. \end{aligned} \tag{3.11}$$

A soma de três senoides vai de -3 a 3. Dividir por 3 e aplicar $x \cdot 0{,}5 + 0{,}5$ normaliza a "energia" e para o intervalo $[0, 1]$. Essa normalização é necessária porque e é usada como peso nas misturas de cor seguintes. A cor final é obtida por interpolação entre uma paleta fixa:

$$C = \mathrm{mix}(c_0, c_1, e), \qquad \mathrm{mix}(a, b, x) = a(1 - x) + b\,x, \tag{3.12}$$

seguida de duas outras misturas controladas por smoothstep, que acendem o ciano e o branco apenas nas regiões de maior energia. Por fim, as linhas de uma grade 8×8 (`fract(uv * 8.0)`) escurecem levemente a cor (25% de mistura com a própria cor reduzida a 35%), e um brilho proporcional a e^3 é somado aos canais verde e azul. Os vetores $\mathbf{k}_i$ definem a direção e a frequência de cada onda, e ω_i a velocidade com que ela percorre a superfície.

3.11 Seleção do nível de mipmap

Onde: unidade de amostragem (sampler), em hardware. **Quando:** a cada leitura de textura.

Quando a textura é reduzida na tela, um pixel cobre vários texels. Heckbert chama a região coberta de pré-imagem do pixel, um quadrilátero curvilíneo no espaço de textura (1, p. 7), e observa que o sistema precisa das derivadas parciais de u e v em relação a x e y da tela, que podem ser aproximadas pela diferença entre pixels vizinhos (1, p. 11). A GPU usa essas derivadas para escolher o nível de detalhe λ:

$$\lambda = \log_2\left(\max\left(\left\|\frac{\partial(u,v)}{\partial x}\right\|, \left\|\frac{\partial(u,v)}{\partial y}\right\|\right) \cdot N\right), \tag{3.13}$$

em que N é a resolução da textura. Com $\lambda = 0$ lê-se a imagem original, com $\lambda = 1$ a imagem reduzida à metade, e assim por diante. Na interpolação trilinear, a GPU lê os dois níveis vizinhos de λ e interpola entre eles, o filtro que Heckbert recomenda como mínimo (1, p. 11). Na implementação atual, a textura é criada com `mipLevels = 1`: existe só o nível 0, e a Equação (3.13) não tem efeito. Essa limitação é discutida no Capítulo 4.

3.12 Iluminação difusa

Onde: fragment shader. **Quando:** uma vez por fragmento, nos modos 1, 2, 4 (parte texturizada) e 5.

$$L = 0{,}35 + 0{,}65\ \max(\hat{\mathbf{n}} \cdot \hat{\mathbf{l}},\ 0), \qquad C_{\text{final}} = C \cdot L. \tag{3.14}$$

No modo 3, o padrão procedural usa uma variante mais clara, $0{,}55 + 0{,}45 \max(\hat{\mathbf{n}} \cdot \hat{\mathbf{l}}, 0)$, para parecer que emite luz própria. No modo 4, as ondas procedurais são misturadas por cima da textura já iluminada, sem nova iluminação.

A normal $\hat{\mathbf{n}}$ é interpolada pelo rasterizador e precisa ser renormalizada (`normalize(fragNormal)`), porque a interpolação de vetores unitários não produz vetores unitários. O termo constante 0,35 é uma luz ambiente que evita que o lado oposto à luz fique completamente preto. A iluminação não faz parte do UV mapping, mas é aplicada por último sobre a cor obtida pela textura ou pela função procedural, para que o volume do modelo continue legível.

3.13 Resumo: equação, estágio e frequência

Tabela 3.1: Em que estágio cada equação é executada e quantas vezes por quadro (janela de 800×600, modelo com 2.399 vértices).

Equação	**Estágio**	**Vezes por quadro**	**Na implementação**
(3.1) UV do vértice	Vertex Input	2.399 a 12.636	`inUV` (location 2)
(3.2) Posição	Vertex Shader	2.399 a 12.636	`gl_Position`
(3.4) Interpolação	Rasterizador	até 480.000	automático
(3.5) Translação	Fragment Shader	até 480.000	`dynamicUV`
(3.6) Ondulação	Fragment Shader	até 480.000	`dynamicUV`
(3.7) Repetição	Sampler	por amostra	`ADDRESS_MODE_REPEAT`
(3.8) Escala	Fragment Shader	até 480.000	`fract(uv * 8.0)`
(3.9) Rotação	CPU + shader	1 + por vértice	não usada
(3.10) Combinada	CPU + shader	1 + por vértice	não usada
(3.11) Procedural	Fragment Shader	até 480.000	`proceduralPattern`
(3.13) Nível de mip-map	Sampler	por amostra	sem efeito (1 nível)
(3.14) Iluminação	Fragment Shader	até 480.000	`main`

Os vértices são 2.399, mas são referenciados por 12.636 índices. Como a GPU reaproveita o resultado de vértices repetidos (*post-transform cache*), o vertex shader roda entre 2.399 e 12.636 vezes por quadro. “Até 480.000” é o número de pixels da janela. O número real de fragmentos depende de quanto da tela o modelo cobre e da sobreposição de triângulos.

4 Relação com o Vulkan e arquitetura do pipeline

4.1 Visão geral

Vulkan é uma API gráfica de baixo nível em que a aplicação declara explicitamente cada recurso que os shaders vão ler, como esses recursos são agrupados, quando são atualizados e como a CPU e a GPU se sincronizam. Para o UV mapping dinâmico e procedural, isso significa responder a quatro perguntas: onde ficam as coordenadas UV, como o tempo chega ao shader a cada quadro, como a textura e o sampler são expostos e onde cada cálculo é feito.

A Figura 4.1 mostra a arquitetura da implementação. Em azul estão os dados e recursos da aplicação; em verde, os estágios do pipeline e os shaders; em laranja, os mecanismos de sincronização; em roxo, o descriptor set, o Render Pass e seus attachments; em amarelo, as operações específicas de UV mapping. As setas contínuas indicam o sentido em que um dado trafega, com o rótulo do que é transmitido. As setas tracejadas em laranja indicam dependências de sincronização. Os grupos tracejados reúnem recursos do mesmo tipo. O arquivo-fonte do diagrama, gerado com Graphviz, está em `docs/diagrama/pipeline_vulkan.dot`. O restante do capítulo descreve cada parte a partir da implementação entregue (`código/src/`, um arquivo por assunto), que renderiza o modelo `Duck.glb` em cinco modos selecionáveis pelas teclas 1 a 5.

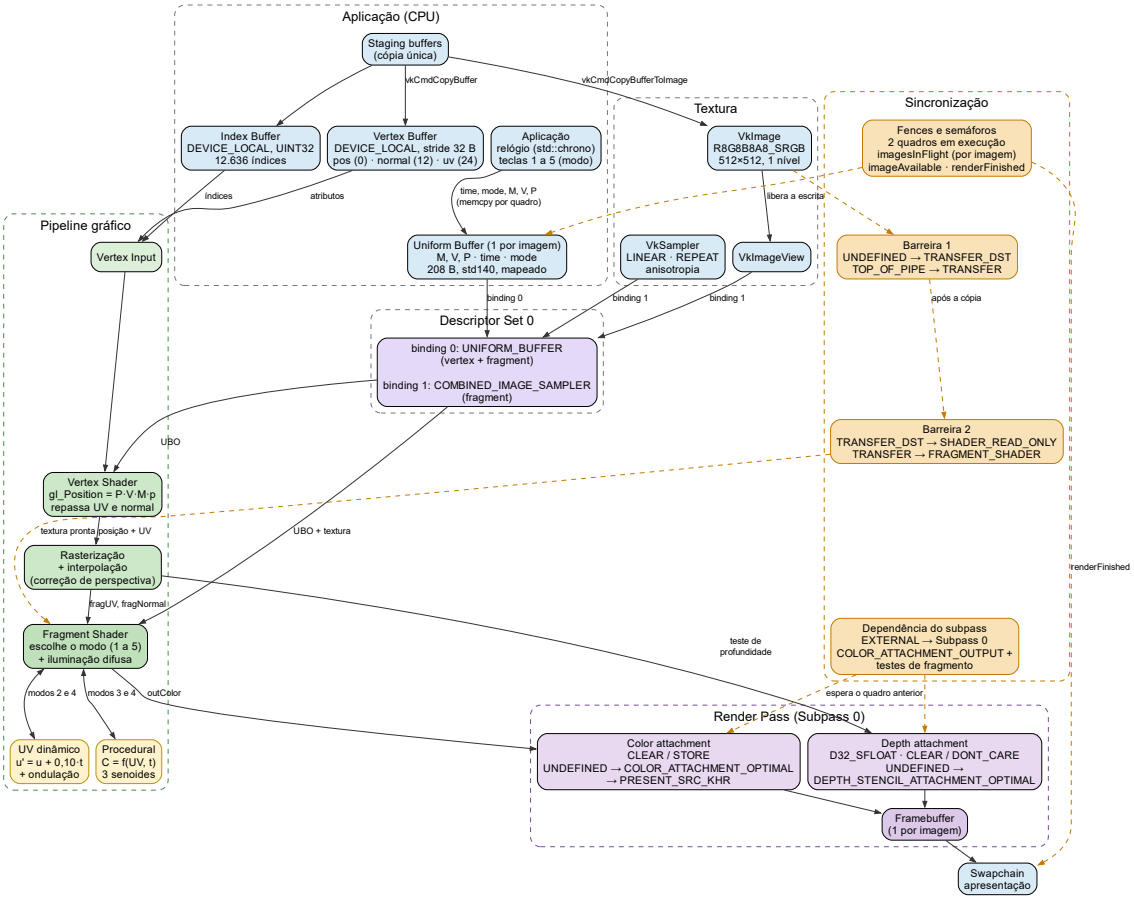


Figura 4.1: Arquitetura do pipeline Vulkan aplicada ao UV mapping dinâmico e procedural, conforme a implementação.

dados e recursos
estágios e shaders
sincronização
descriptor set e render pass
operações de UV mapping
⟶ / ⇢ fluxo de dados / sincronização

4.2 Recursos usados pela técnica

Tabela 4.1: Recursos Vulkan da implementação e o papel de cada um no UV mapping.

Recurso	**Configuração**	**Conteúdo**	**Atualização**
Vertex Buffer	`DEVICE_LOCAL`, enviado via *staging buffer*	2.399 vértices de 32 bytes (posição, normal, UV)	Uma vez, na inicialização
Index Buffer	`DEVICE_LOCAL`, `UINT32`	12.636 índices (4.212 triângulos)	Uma vez
Uniform Buffer	`HOST_VISIBLE | HOST_COHERENT`, mapeado permanentemente; um por imagem da swapchain	208 bytes: M, V, P, `time`, `mode`	A cada quadro (`memcpy`)
Imagem de textura	`R8G8B8A8_SRGB`, 512×512, `mipLevels = 1`	Cor base do modelo	Uma vez
Sampler	`LINEAR`, `REPEAT` nos três eixos, anisotropia no máximo do dispositivo	Filtragem e endereçamento	Imutável

O Vertex Buffer usa um único binding (0), com *stride* de 32 bytes e taxa por vértice. Cada vértice tem três atributos: posição em `location 0` (`R32G32B32_SFLOAT`, offset 0), normal em `location 1` (`R32G32B32_SFLOAT`, offset 12) e UV em `location 2` (`R32G32_SFLOAT`, offset 24). O Index Buffer é uma lista contínua de inteiros de 32 bits, três por triângulo.

A divisão segue a frequência de atualização de cada dado. Geometria e textura nunca mudam e ficam em memória `DEVICE_LOCAL`, a mais rápida para a GPU ler. Os parâmetros que mudam a cada quadro ficam em memória visível pela CPU. O formato `SRGB` faz a GPU converter a cor para o espaço linear na leitura, de modo que a filtragem e a iluminação operam sobre valores lineares. A textura é formada por três objetos (4):

- a `VkImage`, que guarda os texels;
- a `VkImageView`, que define como interpretá-los;
- o `VkSampler`, que define como lê-los.

Para o UV dinâmico, o sampler é tão importante quanto o shader: o modo `REPEAT` permite que $u + 0{,}10\,t$ continue lendo a textura depois de ultrapassar 1, e a filtragem `LINEAR` evita que o deslocamento lento da textura "salte" de um texel para o próximo.

4.3 Fluxo de dados entre os estágios

Tabela 4.2: Estágios do pipeline: o que entra, o que sai e quais recursos são compartilhados.

Estágio	**Entra**	**Sai**	**Recursos usados**
Aplicação (CPU)	Tempo do relógio, tecla do modo	M, V, P, `time`, `mode`	Uniform Buffer da imagem atual
Vertex Input	Vertex e Index Buffers	Posição, normal e UV de cada vértice	Vertex Buffer (32 bytes por vértice), Index Buffer
Vertex Shader	Atributos do vértice, matrizes	`gl_Position`, `fragUV`, `fragNormal`	Binding 0 (UBO)
Rasterização	Triângulos em espaço de recorte	Fragmentos com UV e normal interpolados (Eq. 3.4)	Depth attachment (teste de profundidade)
Fragment Shader	`fragUV`, `fragNormal`, `time`, `mode`, textura	Cor final (`outColor`)	Binding 0 (UBO), binding 1 (textura e sampler)
Saída de cor	Cor do fragmento	Pixel no color attachment	Color attachment, framebuffer, swapchain

Dois recursos são compartilhados entre estágios: o Uniform Buffer, lido pelos dois shaders, e o depth attachment, escrito e testado durante a rasterização. A textura só é lida pelo fragment shader.

4.4 Descriptor Sets e Bindings

Um *descriptor* é a forma como o Vulkan informa a um shader onde está um recurso. Os descriptors são agrupados em *descriptor sets*, cujo formato é declarado por um `VkDescriptorSetLayout`. A implementação usa um único set (`set = 0`) com dois bindings, criados em `createDescriptorSetLayout`.

Tabela 4.3: Layout do Descriptor Set 0 na implementação.

Binding	**Tipo**	**Estágios**	**Declaração GLSL**
0	`UNIFORM_BUFFER`	Vertex e Fragment	`uniform Parameters {...} params`
1	`COMBINED_IMAGE_SAMPLER`	Fragment	`uniform sampler2D baseTexture`

O binding 0 é visível nos dois estágios porque o vertex shader usa as matrizes e o

fragment shader usa `time` e `mode`. O binding 1 é declarado só para o fragment shader, o único que lê a textura. Restringir os `stageFlags` ao necessário informa ao driver quais estágios precisam enxergar cada recurso. São alocados tantos descriptor sets quantas são as imagens da swapchain, cada um apontando para o seu próprio Uniform Buffer. Assim, a CPU não sobrescreve o `time` de um quadro enquanto a GPU ainda lê o anterior. O set de cada imagem é ligado uma única vez, com `vkCmdBindDescriptorSets`, no command buffer daquela imagem.

4.4.1 Do projeto à implementação

O projeto inicial da arquitetura, feito antes da implementação junto com o pseudocódigo, previa três bindings: o Uniform Buffer (0), a imagem (`SAMPLED_IMAGE`, 1) e o sampler (`SAMPLER`, 2), separados. Na implementação, imagem e sampler foram unidos em um `COMBINED_IMAGE_SAMPLER`. As duas formas são válidas, e a escolha depende de quantas texturas existem. Com uma textura e um sampler, o descriptor combinado é mais simples: um binding e uma escrita de descriptor a menos, e o GLSL usa `sampler2D` diretamente. A separação compensa quando muitas texturas compartilham as mesmas regras de filtragem, por exemplo em um material com cor, normal e rugosidade, em que um único `VkSampler` é reutilizado.

O Uniform Buffer também mudou. O projeto previa os campos `uvVelocity` e `uvScale`. A implementação manteve `time`, trocou esses dois campos por `mode` e deixou as velocidades como constantes no shader, porque elas não variam em tempo de execução. O campo `mode` seleciona qual dos cinco cálculos o fragment shader executa.

4.4.2 Layout de memória do Uniform Buffer (std140)

O GLSL usa o layout `std140` para blocos `uniform`, e a struct C++ precisa ter os mesmos deslocamentos em bytes. A implementação verifica isso em tempo de compilação com `static_assert`:

```
struct UniformBufferObject {
    glm::mat4 model;       // offset   0
    glm::mat4 view;        // offset  64
    glm::mat4 projection;  // offset 128
    float     time;        // offset 192
    int32_t   mode;        // offset 196
    float     padding[2];  // total: 208 bytes
};
```

Se um campo fosse reordenado na struct sem a mesma alteração no shader, o tempo

seria lido como lixo e a animação pararia, sem nenhum erro de validação. A verificação transforma esse problema em erro de build.

4.5 Push constants

Push constants são uma segunda forma de enviar dados ao shader, sem buffer nem descriptor. Os valores são gravados diretamente no command buffer com `vkCmdPushConstants` (5) e declarados no shader como `layout(push_constant)`. O espaço é pequeno: o limite `maxPushConstantsSize` tem mínimo garantido de 128 bytes, ou 256 bytes em dispositivos com Vulkan 1.4 (6, 7). Em troca, a atualização dispensa memória mapeada e um buffer por quadro em execução.

Os dados que mais mudam no UV dinâmico são os candidatos ideais. O par `time` e `mode` ocupa 8 bytes; as três matrizes (192 bytes) não cabem no mínimo garantido. Com push constants, o shader e a aplicação ficariam assim:

```
layout(push_constant) uniform Animation { float time; int mode; } anim;

vkCmdPushConstants(cmd, pipelineLayout, VK_SHADER_STAGE_FRAGMENT_BIT,
                   0, sizeof(Animation), &anim);
```

A implementação mantém tudo no Uniform Buffer por uma razão concreta. Os command buffers são gravados uma única vez, em `createCommandBuffers`, um por imagem da swapchain, e depois apenas reenviados. Como o valor de uma push constant fica gravado dentro do command buffer, usá-las exigiria regravá-lo a cada quadro. Com o Uniform Buffer, os command buffers continuam estáticos e só os 208 bytes do buffer mapeado são reescritos. Para uma versão seguinte, a divisão mais adequada é híbrida: matrizes no Uniform Buffer e `time`/`mode` em push constants, com o command buffer regravado por quadro. Essa é a abordagem mais comum em cenas que mudam continuamente, e o custo de gravar um único *draw call* é desprezível.

4.6 Render Pass e Subpass

A implementação usa um Render Pass com um único subpass (Subpass 0), que desenha o modelo, e dois attachments:

- **Color attachment** (formato da swapchain): é limpo no início (`LOAD_OP_CLEAR`), gravado no fim (`STORE_OP_STORE`) e entregue em `PRESENT_SRC_KHR` para ser exibido. Recebe a cor final do fragment shader.
- **Depth attachment** (`D32_SFLOAT` ou o melhor formato disponível): é limpo a cada

quadro e descartado no fim (`STORE_OP_DONT_CARE`), porque a profundidade só serve para decidir a visibilidade dentro do próprio quadro.

Os dois attachments são ligados a um Framebuffer por imagem da swapchain. Um único subpass basta porque a técnica não precisa de pós-processamento: todo o UV mapping acontece no fragment shader do mesmo passo.

4.7 Sincronização: barreiras, layouts e dependências

Transições de layout e barreiras da textura. A textura passa por duas transições de layout, cada uma com uma *image memory barrier*, criadas em `transitionImageLayout`:

Tabela 4.4: Barreiras de memória da textura na implementação.

Campo	**Barreira 1**	**Barreira 2**
Recurso	Imagem da textura	Imagem da textura
Etapa anterior	`TOP_OF_PIPE` (nenhuma)	`TRANSFER`
Acesso anterior	nenhum	escrita de transferência
Etapa posterior	`TRANSFER`	`FRAGMENT_SHADER`
Acesso posterior	escrita de transferência	leitura no shader
Layout anterior	`UNDEFINED`	`TRANSFER_DST_OPTIMAL`
Layout posterior	`TRANSFER_DST_OPTIMAL`	`SHADER_READ_ONLY_OPTIMAL`
Motivo	Colocar a imagem no layout que aceita a cópia do staging buffer (`vkCmdCopyBufferToImage`).	Garantir que a cópia terminou e está visível antes da primeira amostragem; sem ela, o shader poderia ler uma textura parcialmente copiada.

Dependência do subpass e transições dos attachments. As transições dos attachments são feitas pelo próprio Render Pass, sem barreiras explícitas, e sincronizadas por uma `VkSubpassDependency` de `VK_SUBPASS_EXTERNAL` para o Subpass 0:

Tabela 4.5: Dependência do subpass e transições dos attachments.

Campo	**Color attachment**	**Depth attachment**
Recurso	Imagem da swapchain	Imagem de profundidade
Etapa anterior	`COLOR_ATTACHMENT_OUTPUT` (quadro anterior)	`LATE_FRAGMENT_TESTS` (quadro anterior)
Acesso anterior	—	escrita de profundidade
Etapa posterior	`COLOR_ATTACHMENT_OUTPUT`	`EARLY_FRAGMENT_TESTS`
Acesso posterior	escrita de cor	escrita de profundidade
Layout anterior	`UNDEFINED`	`UNDEFINED`
Layout no subpass	`COLOR_ATTACHMENT_OPTIMAL`	`DEPTH_STENCIL_ATTACHMENT_OPTIMAL`
Layout final	`PRESENT_SRC_KHR`	`DEPTH_STENCIL_ATTACHMENT_OPTIMAL`
Motivo	Não escrever na imagem antes de ela ser liberada pela apresentação anterior	Não limpar a profundidade enquanto o quadro anterior ainda a testa e grava

O layout anterior `UNDEFINED` indica que o conteúdo antigo pode ser descartado, o que é correto porque os dois attachments são limpos no início do passo.

Dependências entre CPU e GPU. A cada quadro, a aplicação:

1. espera a fence do quadro em execução (`MAX_FRAMES_IN_FLIGHT = 2`);
2. adquire a próxima imagem da swapchain, com um semáforo (`imageAvailable`) que fará a GPU esperar a imagem ficar livre;
3. espera a fence associada àquela imagem (`imagesInFlight`). Como o Uniform Buffer é um por imagem, é essa espera que garante que a GPU terminou de ler o buffer que será reescrito;
4. mede o tempo e copia M, V, P, `time` e `mode` para o Uniform Buffer daquela imagem (`updateUniformBuffer`);
5. submete o command buffer, que sinaliza o semáforo `renderFinished`, esperado pela apresentação.

A escrita da CPU fica visível para a GPU sem barreira explícita porque a memória é `HOST_COHERENT` e a submissão à fila garante a ordem entre a escrita no host e as leituras dos shaders.

4.8 Onde cada cálculo é feito

Em Vulkan, a aplicação escolhe se um cálculo roda na CPU (uma vez por quadro), no vertex shader (por vértice) ou no fragment shader (por pixel). Com 2.399 vértices contra

até 480.000 pixels (Capítulo 3), essa é a decisão de desempenho mais importante da técnica.

- **CPU:** medir o tempo, montar M, V, P e, se houvesse rotação de UV, calcular $\cos\theta$ e $\sin\theta$.
- **Vertex shader:** transformações lineares do UV (translação, escala, rotação), que interpoladas dão o mesmo resultado que se fossem feitas por fragmento.
- **Fragment shader:** tudo o que é não linear (ondulação, padrão procedural, grade) e a amostragem da textura.

O projeto fazia a translação no vertex shader. A implementação a moveu para o fragment shader, dentro de `dynamicUV`, junto com a ondulação. Isso mantém todo o efeito em uma função legível, ao custo de uma soma por pixel em vez de uma por vértice, diferença desprezível neste caso.

4.9 Otimizações: aplicadas e pendentes

Tabela 4.6: Otimizações relevantes para a técnica e sua situação na implementação.

Otimização	**Situação**	**Efeito**
Geometria em `DEVICE_LOCAL` via staging	Aplicada	A GPU lê vértices e índices da memória mais rápida; a cópia é feita uma vez com `vkCmdCopyBuffer`.
Uniform Buffer mapeado permanentemente	Aplicada	Sem `vkMapMemory`/`vkUnmapMemory` por quadro: a atualização é um `memcpy` de 208 bytes.
Um Uniform Buffer por imagem e 2 quadros em execução	Aplicada	A CPU prepara o próximo quadro enquanto a GPU desenha o atual, sem escrever em um buffer em uso.
Filtragem anisotrópica	Aplicada	Reduz o borrão em superfícies inclinadas; ativada só se o dispositivo suportar.
Command buffers pré-gravados	Aplicada	Nenhum custo de gravação por quadro; é o motivo de `time` estar no Uniform Buffer.
Mipmaps	**Pendente**	A textura tem `mipLevels = 1`, o que causa aliasing com o modelo pequeno na tela. A correção é criar $\lfloor \log_2 512 \rfloor + 1 = 10$ níveis e gerá-los com `vkCmdBlitImage` (8), ao custo de 1/3 a mais de memória (1, p. 9).
Specialization constants por modo	Pendente	O shader escolhe o modo com `if` sobre `params.mode`. Não há divergência entre threads, pois o valor é igual para todos os fragmentos, mas o shader contém os cinco caminhos. Com `VkSpecializationInfo`, cada modo viraria um pipeline só com o código necessário.
Transformações lineares no vertex shader	Pendente	A translação de `dynamicUV` poderia rodar de 2.399 a 12.636 vezes por quadro, em vez de até 480.000.
Índices de 16 bits	Pendente	O glTF guarda os índices em 16 bits, e a implementação os converte para 32. Com menos de 65.536 vértices, `UINT16` reduziria o Index Buffer à metade.
Tempo periódico na CPU	Pendente	Enviar $t \bmod T$, calculado em precisão dupla na CPU, evita a perda de precisão do `float` após cerca de 18 horas. Para não haver salto na volta, os movimentos precisam se repetir em T (Seção 2.5).

4.10 Justificativa das decisões arquiteturais

- **Um único subpass:** todo o efeito acontece no fragment shader do mesmo passo. Não há pós-processamento que justifique um segundo subpass ou render pass.
- **Geometria e textura em memória `DEVICE_LOCAL`:** são enviadas uma vez e lidas em todos os quadros, então pagam o custo do staging uma única vez.
- **Parâmetros em um Uniform Buffer por imagem da swapchain:** mantêm os command buffers estáticos e evitam escrever em um buffer que a GPU ainda está lendo.
- **Sampler em `REPEAT` com filtragem linear:** é o que torna possível deslocar o UV indefinidamente sem costura nem saltos de texel.
- **Cálculo do efeito no fragment shader:** a ondulação e o padrão procedural são não lineares e precisam ser avaliados por pixel. A translação, que é linear, foi mantida junto delas por legibilidade.
- **Descriptor combinado:** com uma única textura, é a organização mais simples e não perde nada em relação a imagem e sampler separados.

5 Shaders: pseudocódigo e implementação

Este capítulo documenta o pseudocódigo dos shaders, entregue em dois arquivos na pasta `docs/pseudocodigo/`: `uv_mapping.vert`, o vertex shader, e `uv_mapping.frag`, o fragment shader. As seções seguem a ordem pedida no enunciado: objetivo, fluxo, pseudocódigo completo, entradas e saídas, equações e variáveis, condições e laços, amostragem, tratamento de limites, resultado esperado e relação com a técnica.

O pseudocódigo usa a sintaxe do GLSL e tem, instrução por instrução, o mesmo código dos shaders compilados da implementação (`código/shaders/`). Os dois diferem apenas nos comentários: os arquivos de pseudocódigo explicam cada trecho. Assim, o que está descrito aqui é exatamente o que roda na aplicação.

5.1 Objetivo de cada shader

- **Vertex shader** (`uv_mapping.vert`): roda uma vez por vértice. Calcula a posição do vértice na tela e repassa o UV e a normal. Ele não altera o UV.
- **Fragment shader** (`uv_mapping.frag`): roda uma vez por fragmento, isto é, para cada pixel coberto por um triângulo do modelo. É onde está o efeito: conforme o modo escolhido, ele lê a textura com o UV original ou deslocado no tempo (UV dinâmico), gera a cor por funções matemáticas (procedural), combina os dois ou desenha uma grade de depuração. Cada modo aplica a sua iluminação.

Não é usado compute shader: todo o efeito depende só do UV e do tempo de cada pixel, sem dados de vizinhos, e por isso cabe no fragment shader.

5.2 Fluxo entre os shaders

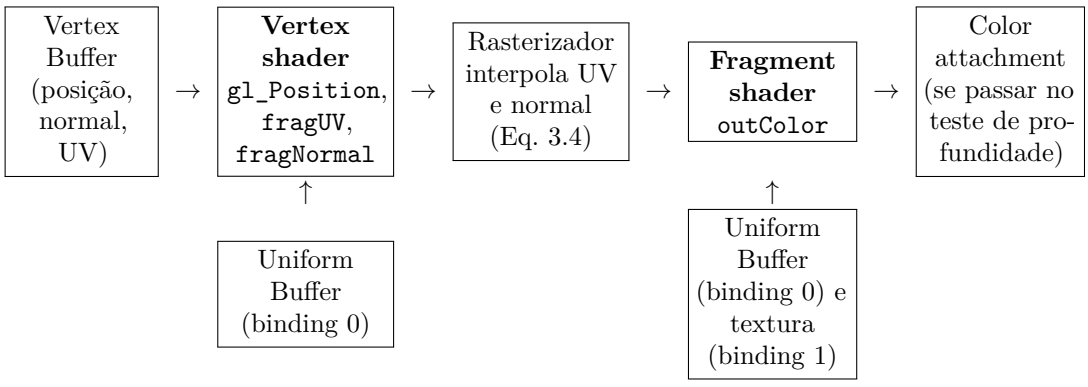


Figura 5.1: Fluxo de dados entre os shaders.

O vertex shader recebe os atributos de cada vértice e as matrizes. Ele entrega a posição

na tela e duas variáveis, `fragUV` e `fragNormal`. O rasterizador interpola essas variáveis dentro de cada triângulo, com correção de perspectiva (Equação (3.4)), e o fragment shader as recebe já com o valor daquele pixel. A cor que ele produz vai para o color attachment. Os dois shaders leem o mesmo Uniform Buffer (binding 0); só o fragment shader lê a textura (binding 1).

5.3 Pseudocódigo completo

Os comentários seguem o mesmo padrão em todos os blocos: **Entra** (o que o trecho recebe), **Faz** (o cálculo e para que serve), **Sai** (o resultado) e **Na tela** (como isso aparece no efeito final). Cada declaração traz o tipo e o significado da variável, e os cálculos que têm equação no Capítulo 3 trazem o número dela (por exemplo, "Eq. 3.5" na translação).

O pseudocódigo foi escrito antes da implementação e ajustado depois dela; as mudanças nos bindings e no Uniform Buffer estão na Seção 4.4.1, e as demais foram:

- a normal passou a ser girada pela matriz `model`;
- o deslocamento do UV saiu do vertex shader e foi para o fragment shader, na mesma função da ondulação (`dynamicUV`). Por ser linear, a translação poderia continuar no vertex shader; a mudança foi feita por legibilidade e para que o `fragUV` original continue disponível nos modos 1, 3 e 5 (Seção 4.8);
- a repetição do UV, antes feita com `fract` no shader, passou para o sampler (`REPEAT`);
- o `clamp` da cor final foi retirado, porque o formato do color attachment já limita a cor;
- a mistura fixa de 50% foi substituída por cinco modos;
- a onda única do padrão procedural virou uma soma de três ondas, com distorção.

5.3.1 Vertex shader (`uv_mapping.vert`)

```
// ============================================================================
// PSEUDOCÓDIGO - VERTEX SHADER (uv_mapping.vert)
// Tema: UV Mapping Dinâmico e Procedural
//
// Objetivo: posicionar cada vértice do modelo na tela e entregar ao
//   rasterizador a coordenada UV e a normal desse vértice.
// Entra:    posição, normal e UV de um vértice (lidos do modelo glTF)
//           e as matrizes model, view e projection (Uniform Buffer).
// Sai:      gl_Position (posição na tela) e fragUV, fragNormal, que o
//           rasterizador interpola e entrega ao fragment shader.
// Roda:     uma vez por vértice, a cada quadro.
//
```

```
// Fluxo no pipeline:
//   Vertex Buffer -> [ESTE SHADER] -> rasterizador (interpola UV e normal)
//   -> fragment shader (uv_mapping.frag) -> cor do pixel
//
// Este shader NÃO muda o UV: ele só o repassa. Todo o efeito dinâmico e
// procedural é calculado por pixel, no fragment shader.
// Padrão dos comentários: Entra / Faz / Sai / Na tela.
// ============================================================================

#version 450

// ----------------------------------------------------------------------------
// RECURSO: Uniform Buffer (set 0, binding 0), o mesmo nos dois shaders.
// A CPU reescreve este bloco a cada quadro (208 bytes, layout std140).
// ----------------------------------------------------------------------------
layout(set = 0, binding = 0) uniform Parameters
{
    mat4 model;       // mat4: gira o modelo (só rotação no eixo Y)
    mat4 view;        // mat4: posição e direção da câmera
    mat4 projection;  // mat4: projeção em perspectiva (com profundidade)
    float time;       // float: segundos desde o início (t nas equações);
                      //   não é usado aqui, só no fragment shader
    int mode;         // int: modo escolhido nas teclas 1 a 5;
                      //   não é usado aqui, só no fragment shader
} params;

// ----------------------------------------------------------------------------
// ENTRADAS: atributos de um vértice, lidos do Vertex Buffer
// (32 bytes por vértice: posição, normal e UV, todos em float).
// ----------------------------------------------------------------------------
layout(location = 0) in vec3 inPosition; // vec3: posição no espaço do modelo
layout(location = 1) in vec3 inNormal;   // vec3: normal, já unitária
                                         //   (o ModelLoader garante isso)
layout(location = 2) in vec2 inUV;       // vec2: coordenada (u, v) na
                                         //   textura, do TEXCOORD_0 do glTF

// ----------------------------------------------------------------------------
// SAÍDAS: vão para o rasterizador, que interpola cada uma dentro do
// triângulo, com correção de perspectiva (Eq. 3.4).
// ----------------------------------------------------------------------------
layout(location = 0) out vec2 fragUV;     // vec2: UV que o pixel vai usar
layout(location = 1) out vec3 fragNormal; // vec3: normal para a iluminação
// gl_Position (vec4) é a saída obrigatória do vertex shader.

void main()
{
    // ------------------------------------------------------------------------
```

```
60      // 1. POSIÇÃO EM ESPAÇO DE RECORTE (Eq. 3.2, transformação da posição)
61      // Entra:  inPosition (vec3) e as três matrizes.
62      // Faz:    p_clip = projection * view * model * p.
63      //         O vec4(..., 1.0) põe a posição em coordenadas homogêneas,
64      //         para que a matriz possa também deslocar o ponto.
65      // Sai:    gl_Position (vec4), em espaço de recorte.
66      // Na tela: o modelo aparece girando, visto pela câmera.
67      // Depois deste shader, o hardware divide gl_Position por w e aplica o
68      //         viewport. O z/w (de 0 a 1) vira a profundidade do pixel e é
69      //         comparado com o depth attachment (teste LESS: fica o ponto
70      //         mais próximo). Este shader não escreve profundidade à mão.
71      // ------------------------------------------------------------------------
72      gl_Position =
73          params.projection *
74          params.view *
75          params.model *
76          vec4(inPosition, 1.0);
77
78      // ------------------------------------------------------------------------
79      // 2. COORDENADA UV (Eq. 3.1, coordenadas UV do vértice)
80      // Entra:  inUV (vec2), entre 0 e 1.
81      // Faz:    copia sem alterar. O rasterizador interpola o valor entre os
82      //         três vértices do triângulo (Eq. 3.4).
83      // Sai:    fragUV (vec2).
84      // Na tela: cada pixel do modelo recebe a sua posição na textura.
85      //         Isso é o mapeamento UV.
86      // ------------------------------------------------------------------------
87      fragUV = inUV;
88
89      // ------------------------------------------------------------------------
90      // 3. NORMAL (operação com matriz)
91      // Entra:  inNormal (vec3) e a matriz model.
92      // Faz:    gira a normal junto com o modelo. mat3(model) pega só a parte
93      //         de rotação da matriz 4x4 (sem o deslocamento). Como model é
94      //         só rotação, ela serve para normais sem precisar da inversa
95      //         transposta, e a normal continua com comprimento 1.
96      // Sai:    fragNormal (vec3).
97      // Na tela: a luz fica parada enquanto o modelo gira; sem esta linha,
98      //         a parte iluminada giraria junto com o pato.
99      // ------------------------------------------------------------------------
100     fragNormal = mat3(params.model) * inNormal;
101 }
```

5.3.2 Fragment shader (`uv_mapping.frag`)

```
// ============================================================================
// PSEUDOCÓDIGO - FRAGMENT SHADER (uv_mapping.frag)
// Tema: UV Mapping Dinâmico e Procedural
//
// Objetivo: calcular a cor de cada ponto do modelo. É aqui que acontece o
//   efeito: o UV é deslocado com o tempo (UV dinâmico) e/ou a cor é gerada
//   por uma função matemática (procedural).
// Entra:    fragUV e fragNormal (vindos do vertex shader, interpolados pelo
//           rasterizador), time e mode (Uniform Buffer) e a textura.
// Sai:      outColor, a cor final do pixel (vec4, RGBA).
// Roda:     uma vez por fragmento (cada pixel coberto por um triângulo do
//           modelo), a cada quadro.
//
// Fluxo dentro do shader:
//   1. calcula a luz (normal + direção da luz)
//   2. escolhe o modo (1 a 5) e calcula a cor desse modo
//   3. escreve outColor
//
// Os 5 modos (tecla 1 a 5 na aplicação):
//   1 textura   2 UV dinâmico   3 procedural   4 mistura   5 grade UV
//
// Laços: NÃO há laços. As três ondas do padrão procedural são uma soma de
//   tamanho fixo, escrita termo a termo. Como não há repetição, não existe
//   critério de parada. Cada fragmento faz sempre o mesmo número de
//   operações.
//
// Padrão dos comentários: Entra / Faz / Sai / Na tela.
// ============================================================================

#version 450

// ----------------------------------------------------------------------------
// RECURSO: Uniform Buffer (set 0, binding 0), o mesmo do vertex shader.
// ----------------------------------------------------------------------------
layout(set = 0, binding = 0) uniform Parameters
{
    mat4 model;       // mat4: rotação do modelo (usada no vertex shader)
    mat4 view;        // mat4: câmera (usada no vertex shader)
    mat4 projection;  // mat4: projeção (usada no vertex shader)
    float time;       // float: segundos desde o início; faz o efeito andar
    int mode;         // int: modo de 1 a 5, igual para todos os pixels
} params;

// ----------------------------------------------------------------------------
```

```
45 // RECURSO: textura + sampler (set 0, binding 1, COMBINED_IMAGE_SAMPLER).
46 // sampler2D: imagem 2D já ligada às regras de leitura (o sampler).
47 // Configuração do sampler, feita na aplicação (createTextureSampler):
48 //   - filtro LINEAR na ampliação e na redução: mistura os 4 texels mais
49 //     próximos (com anisotropia, pode combinar mais amostras), para o
50 //     deslocamento lento não "pular" de texel em texel;
51 //   - endereçamento REPEAT em u e v: UV acima de 1 volta para o começo
52 //     (Eq. 3.7). É o que deixa o UV crescer sem limite no modo 2;
53 //   - mipmap LINEAR, mas a imagem tem 1 só nível (mipLevels = 1), então a
54 //     escolha de nível (Eq. 3.13) não tem efeito;
55 //   - anisotropia ligada só se a GPU suportar, no máximo que ela permite.
56 // A imagem é R8G8B8A8_SRGB: a leitura já devolve a cor em espaço linear.
57 // ------------------------------------------------------------------------
58 layout(set = 0, binding = 1) uniform sampler2D baseTexture;
59
60 // ------------------------------------------------------------------------
61 // ENTRADAS: saídas do vertex shader, já interpoladas para este pixel.
62 // ------------------------------------------------------------------------
63 layout(location = 0) in vec2 fragUV;     // vec2: coordenada na textura
64 layout(location = 1) in vec3 fragNormal; // vec3: normal; a interpolação
65                                          //   pode deixá-la menor que 1
66
67 // ------------------------------------------------------------------------
68 // SAÍDA: cor do pixel, gravada no color attachment (location 0).
69 // ------------------------------------------------------------------------
70 layout(location = 0) out vec4 outColor;  // vec4: vermelho, verde, azul, alfa
71
72 // Modos (mesmos valores de RenderMode na aplicação, VulkanApp.hpp)
73 const int MODE_TEXTURE = 1;     // textura original
74 const int MODE_DYNAMIC_UV = 2;  // textura com UV dinâmico
75 const int MODE_PROCEDURAL = 3;  // cor procedural, sem imagem
76 const int MODE_BLEND = 4;       // UV dinâmico + procedural (modo inicial)
77 const int MODE_UV_GRID = 5;     // grade para ver o próprio UV
78
79
80 // ========================================================================
81 // UV DINÂMICO: dynamicUV (Eq. 3.5 translação e Eq. 3.6 ondulação)
82 // Entra:  uv (vec2, o UV do pixel) e time (float, segundos).
83 // Faz:    1) soma 0,10 ao u a cada segundo: a textura dá uma volta
84 //            completa sobre o modelo a cada 10 segundos;
85 //         2) soma uma ondulação pequena (amplitude 0,015), diferente em
86 //            cada região, que dá a impressão de líquido.
87 //         A linha de v usa o u que acabou de ser alterado (Eq. 3.6).
88 // Sai:    o UV deslocado (vec2). Pode passar de 1: o sampler REPEAT trata.
89 // Na tela: a malha não se deforma; a textura escorre e ondula sobre ela.
90 // ========================================================================
91 vec2 dynamicUV(vec2 uv, float time)
```

```
92 {
93     uv.x += time * 0.10;                              // translação (Eq. 3.5)
94
95     uv.x += sin(uv.y * 12.0 + time * 2.0) * 0.015;  // ondulação em u
96     uv.y += cos(uv.x * 12.0 + time * 1.5) * 0.015;  // ondulação em v
97
98     return uv;
99 }
100
101
102 // ============================================================================
103 // PADRÃO PROCEDURAL: proceduralPattern (Eq. 3.11 e Eq. 3.12)
104 // Entra:  baseUV (vec2) e time (float).
105 // Faz:    gera a cor só com cálculos, sem ler imagem nenhuma, em 6 passos.
106 // Sai:    vec4: rgb = cor do padrão; a = energy (0 a 1), a "força" da
107 //         onda naquele pixel, usada depois como máscara no modo 4.
108 // Na tela: ondas azuis e ciano que andam pela superfície.
109 // ============================================================================
110 vec4 proceduralPattern(vec2 baseUV, float time)
111 {
112     vec2 uv = baseUV;  // vec2: cópia do UV que vai ser deformada
113
114     // Passo 1 - movimento: o padrão anda em u e balança em v.
115     uv.x += time * 0.40;
116     uv.y += sin(time * 1.2) * 0.10;
117
118     // Passo 2 - distorção (amplitude 0,08): entorta o uv para as ondas não
119     // ficarem retas. Na tela: as ondas ficam irregulares, menos retas.
120     // distortionX e distortionY são calculadas com o mesmo uv e só depois
121     // somadas.
122     float distortionX = sin(uv.y * 14.0 + time * 2.0) * 0.08;  // desvio u
123     float distortionY = cos(uv.x * 12.0 - time * 1.5) * 0.08;  // desvio v
124     uv.x += distortionX;
125     uv.y += distortionY;
126
127     // Passo 3 - três ondas planas (Eq. 3.11). Cada uma é um seno de
128     // (k . uv + w * t): k dá a direção e a frequência, w a velocidade.
129     // Cada wave (float) vai de -1 a 1.
130     float wave1 = sin(uv.x * 20.0 + uv.y * 8.0 + time * 4.0);  // k=(20,8)
131     float wave2 = sin(uv.y * 24.0 - uv.x * 6.0 - time * 3.0);  // k=(-6,24)
132     float wave3 = sin((uv.x + uv.y) * 18.0 + time * 2.0);      // k=(18,18)
133
134     // Passo 4 - energia: a soma vai de -3 a 3. Dividir por 3 leva para
135     // [-1, 1], e "* 0.5 + 0.5" leva para [0, 1]. Isso é necessário porque
136     // energy vira peso de mix e smoothstep, que esperam valores de 0 a 1.
137     float energy = (wave1 + wave2 + wave3) / 3.0;  // float, de -1 a 1
138     energy = energy * 0.5 + 0.5;                   // float, de 0 a 1
```

```
139
140     // Passo 5 - grade 8x8 (Eq. 3.8, escala). fract(uv * 8.0) repete o
141     // intervalo [0, 1] oito vezes. grid (float) vale 1 perto das linhas
142     // da grade e 0 no meio das células; smoothstep suaviza a borda.
143     vec2 gridUV = fract(uv * 8.0);  // vec2: posição dentro da célula
144     float gridX = smoothstep(0.46, 0.50, abs(gridUV.x - 0.5));
145     float gridY = smoothstep(0.46, 0.50, abs(gridUV.y - 0.5));
146     float grid = max(gridX, gridY);
147
148     // Passo 6 - cor (Eq. 3.12, mix). Paleta fixa de quatro cores (vec3):
149     vec3 deepBlue = vec3(0.005, 0.015, 0.12);  // energia baixa
150     vec3 blue = vec3(0.0, 0.18, 0.85);
151     vec3 cyan = vec3(0.0, 0.85, 1.0);
152     vec3 white = vec3(0.65, 1.0, 1.0);         // energia máxima
153
154     // mix(a, b, x) = a * (1 - x) + b * x. O ciano só aparece com energy
155     // acima de 0,55 e o branco acima de 0,90 (smoothstep).
156     vec3 color = mix(deepBlue, blue, energy);
157     color = mix(color, cyan, smoothstep(0.55, 0.90, energy));
158     color = mix(color, white, smoothstep(0.90, 1.0, energy));
159
160     // Linhas da grade: mistura 25% com a cor escurecida para 35%.
161     color = mix(color, color * 0.35, grid * 0.25);
162
163     // Brilho nas cristas: soma até 0,25 no verde e 0,35 no azul.
164     // A cor pode passar de 1 aqui (ver SAÍDA, no fim do main).
165     float glow = pow(energy, 3.0);  // float: cresce rápido perto de 1
166     color += vec3(0.0, 0.25, 0.35) * glow;
167
168     return vec4(color, energy);
169 }
170
171
172 // ============================================================================
173 // GRADE UV: uvGrid (depuração, Eq. 3.8 escala)
174 // Entra:  uv (vec2).
175 // Faz:    pinta u em vermelho e v em verde, desenha um xadrez 10x10 e
176 //         linhas brancas nas bordas das células.
177 // Sai:    cor (vec3).
178 // Na tela: mostra como a textura foi esticada sobre o modelo e onde ficam
179 //         as costuras do UV.
180 // ============================================================================
181 vec3 uvGrid(vec2 uv)
182 {
183     vec2 cells = uv * 10.0;  // vec2: 10 células em cada direção
184
185     // checker (float): 0 ou 1, alterna como um tabuleiro de xadrez
```

```
186     float checker = mod(floor(cells.x) + floor(cells.y), 2.0);
187
188     // line (float): 1 na borda da célula, 0 no meio
189     vec2 lineDistance = abs(fract(cells) - 0.5);
190     float line = smoothstep(0.44, 0.48, max(lineDistance.x, lineDistance.y));
191
192     // fract mantém u e v entre 0 e 1, para virarem cor válida
193     vec3 color = vec3(fract(uv.x), fract(uv.y), 0.2);
194     color *= 0.65 + 0.35 * checker;  // células alternadas mais escuras
195
196     return mix(color, vec3(1.0), line);  // linhas em branco
197 }
198
199
200 void main()
201 {
202     float time = params.time;  // float: t, em segundos
203
204     // ========================================================================
205     // 1. ILUMINAÇÃO DIFUSA (Eq. 3.14)
206     // Entra:  fragNormal (vec3).
207     // Faz:    normalize devolve a normal ao comprimento 1, porque a
208     //         interpolação encurta o vetor. dot(normal, luz) é o cosseno do
209     //         ângulo entre os dois; max(..., 0.0) zera o lado de trás.
210     //         lighting soma 0,35 de luz ambiente, para nada ficar preto.
211     // Sai:    diffuse (float, 0 a 1) e lighting (float, 0,35 a 1).
212     // Na tela: o modelo tem um lado claro e um lado escuro, e dá para ver
213     //         o volume.
214     // ========================================================================
215     vec3 normal = normalize(fragNormal);                 // vec3, comprimento 1
216     vec3 lightDirection = normalize(vec3(0.4, 0.8, 0.6)); // vec3: luz fixa
217     float diffuse = max(dot(normal, lightDirection), 0.0);
218     float lighting = 0.35 + 0.65 * diffuse;
219
220     vec3 color;  // vec3: cor do pixel antes da saída
221
222     // ========================================================================
223     // 2. ESCOLHA DO MODO (condição)
224     // params.mode é igual para todos os pixels do quadro, então todos
225     // seguem o mesmo ramo (sem divergência entre threads da GPU).
226     // Cada ramo aplica a sua própria iluminação.
227     // ========================================================================
228     if (params.mode == MODE_TEXTURE)
229     {
230         // Modo 1. Faz: lê a textura no UV original (amostragem com o
231         // sampler LINEAR/REPEAT) e aplica a luz.
232         // Na tela: o modelo com a textura original, parada.
```

```
233         color = texture(baseTexture, fragUV).rgb * lighting;
234     }
235     else if (params.mode == MODE_DYNAMIC_UV)
236     {
237         // Modo 2. Faz: lê a textura no UV deslocado por dynamicUV.
238         // Na tela: a textura desliza e ondula; a malha não se deforma.
239         color = texture(baseTexture, dynamicUV(fragUV, time)).rgb * lighting;
240     }
241     else if (params.mode == MODE_PROCEDURAL)
242     {
243         // Modo 3. Faz: usa só o padrão procedural, sem textura.
244         // Luz mais clara (0,55 + 0,45), para o padrão parecer emitir luz.
245         vec4 pattern = proceduralPattern(fragUV, time);  // rgb + energy
246         color = pattern.rgb * (0.55 + 0.45 * diffuse);
247     }
248     else if (params.mode == MODE_UV_GRID)
249     {
250         // Modo 5. Faz: desenha a grade de depuração com a luz normal.
251         color = uvGrid(fragUV) * lighting;
252     }
253     else
254     {
255         // Modo 4 (MODE_BLEND) e qualquer valor inválido (fora de 1 a 5):
256         // cai aqui, então o shader nunca fica sem cor.
257         // Faz: textura com UV dinâmico + ondas procedurais por cima.
258         // waveMask (float, 0 a 0,75): só as regiões de energia alta (acima
259         // de 0,45) mostram as ondas, e nunca cobrem mais de 75%.
260         // A parte procedural entra sem nova iluminação.
261         // Na tela: a textura escorrendo com ondas de luz passando por cima.
262         vec3 textured =
263             texture(baseTexture, dynamicUV(fragUV, time)).rgb * lighting;
264         vec4 pattern = proceduralPattern(fragUV, time);
265         float waveMask = smoothstep(0.45, 0.95, pattern.a) * 0.75;
266         color = mix(textured, pattern.rgb, waveMask);
267     }
268
269     // ========================================================================
270     // 3. SAÍDA
271     // Entra:  color (vec3).
272     // Faz:    junta a cor com alfa 1 (opaco). Não há clamp no shader: se o
273     //         brilho passar de 1, o color attachment limita cada canal a
274     //         [0, 1] ao gravar o pixel, porque usa um formato normalizado
275     //         de 8 bits (a aplicação prefere B8G8R8A8_SRGB).
276     // Sai:    outColor (vec4), a cor final do pixel.
277     // ========================================================================
278     outColor = vec4(color, 1.0);
279 }
```

```
280
281 // ============================================================================
282 // CASOS EXTREMOS E VALORES INVÁLIDOS (resumo)
283 // - UV fora de [0, 1]: o modo 2 faz u crescer sem limite. O sampler REPEAT
284 //   volta para o começo da textura (Eq. 3.7). Na grade, fract faz isso.
285 // - Normal nula: normalize de um vetor zero não tem resultado válido. O
286 //   carregador do modelo (ModelLoader.cpp) calcula as normais que faltam
287 //   pela média das faces e troca normais de comprimento quase zero por
288 //   (0, 1, 0). Assim, nenhum vértice chega aqui com normal nula. Dentro
289 //   do triângulo, a interpolação só chegaria perto de zero se os vértices
290 //   tivessem normais quase opostas, o que não ocorre numa malha suave.
291 // - Luz por trás: max(..., 0.0) impede luz negativa, e o 0,35 de ambiente
292 //   evita preto total.
293 // - Energia procedural: normalizada para [0, 1] antes de virar peso.
294 // - Cor acima de 1: limitada pelo formato do color attachment na escrita.
295 // - Modo inválido: cai no último ramo (else), que é a mistura.
296 // - Tempo muito grande: time é float. Depois de cerca de 18 horas, o
297 //   menor passo de time chega a 0,008 s (quase meio quadro) e o movimento
298 //   fica em degraus; o UV deslocado (u + 0,10 t e u + 0,40 t) também
299 //   perde precisão. Não é tratado no shader. A correção possível, na
300 //   aplicação, está em "Limitações", no Capítulo 2 do PDF.
301 // ============================================================================
```

5.4 Entradas, saídas e recursos

Tabela 5.1: Entradas, saídas e recursos dos shaders.

Nome	Tipo	Origem	Significado
`inPosition`	`vec3`	vértice, location 0	Posição do vértice no espaço do modelo
`inNormal`	`vec3`	vértice, location 1	Normal do vértice, unitária
`inUV`	`vec2`	vértice, location 2	Coordenada na textura (`TEXCOORD_0` do glTF), entre 0 e 1
`params.model`	`mat4`	binding 0	Rotação do modelo no eixo Y
`params.view`, `projection`	`mat4`	binding 0	Câmera e projeção em perspectiva
`params.time`	`float`	binding 0	Segundos desde o início (t nas equações)
`params.mode`	`int`	binding 0	Modo de renderização, de 1 a 5
`baseTexture`	`sampler2D`	binding 1	Textura do modelo com o sampler (`LINEAR`, `REPEAT`)
`gl_Position`	`vec4`	saída do vertex	Posição em espaço de recorte; depois da divisão por w, z/w vira a profundidade do pixel
`fragUV`	`vec2`	vertex → fragment	UV do pixel, interpolado
`fragNormal`	`vec3`	vertex → fragment	Normal do pixel, interpolada (precisa ser renormalizada)
`outColor`	`vec4`	saída do fragment	Cor final do pixel, RGBA (location 0)

5.5 Equações e variáveis

A Tabela 5.2 liga cada cálculo do pseudocódigo à sua equação do Capítulo 3 e às variáveis locais que ele usa. As funções do GLSL que aparecem nos cálculos são:

- `mix(a, b, x)` $= a(1-x) + b\,x$, uma mistura entre a e b;
- `smoothstep(e0, e1, x)`: vale 0 abaixo de e_0, 1 acima de e_1, e faz uma transição suave entre os dois;
- `fract(x)` $= x - \lfloor x \rfloor$, a parte fracionária;
- `normalize(v)` $= v/\|v\|$, o vetor com comprimento 1;
- `dot(a, b)`, o produto escalar, que para vetores unitários é o cosseno do ângulo entre

eles.

Tabela 5.2: Cálculos do pseudocódigo, equações e variáveis.

Cálculo (onde)	**Equação**	**Variáveis (tipo)**	**Resultado**
Posição (`.vert`, passo 1)	(3.2) transformação	`inPosition` (`vec3`), matrizes (`mat4`)	`gl_Position`
UV (`.vert`, passo 2)	(3.1) UV do vértice	`inUV` (`vec2`)	`fragUV`
Normal (`.vert`, passo 3)	rotação por `mat3(model)`	`inNormal` (`vec3`)	`fragNormal`
`dynamicUV`	(3.5) translação e (3.6) ondulação	`uv` (`vec2`), `time` (`float`)	UV deslocado (`vec2`)
`proceduralPattern`, passos 1 e 2	deslocamento e distorção	`distortionX`, `distortionY` (`float`)	UV deformado
`proceduralPattern`, passos 3 e 4	(3.11) três ondas	`wave1`–`wave3`, `energy` (`float`)	energia em $[0, 1]$
`proceduralPattern`, passo 5	(3.8) escala	`gridUV` (`vec2`), `grid` (`float`)	linhas da grade 8×8
`proceduralPattern`, passo 6	(3.12) mistura	`deepBlue`, `blue`, `cyan`, `white`, `color` (`vec3`); `glow` (`float`)	cor e energia (`vec4`)
`uvGrid`	(3.8) escala	`cells`, `lineDistance` (`vec2`); `checker`, `line` (`float`)	cor da grade (`vec3`)
Iluminação (`main`, passo 1)	(3.14) difusa	`normal`, `lightDirection` (`vec3`); `diffuse`, `lighting` (`float`)	fator de luz
Modo 4 (`main`, passo 2)	(3.12) mistura	`textured` (`vec3`), `pattern` (`vec4`), `waveMask` (`float`)	cor misturada
Leitura da textura	(3.7) repetição e (3.13) mipmap	UV (`vec2`), `baseTexture`	cor do texel

5.6 Condições e laços

O fragment shader tem uma única cadeia de condições, sobre `params.mode`, que escolhe um de cinco cálculos (passo 2 do `main`). Como `mode` vem do Uniform Buffer, o valor é o mesmo para todos os pixels do quadro: todas as threads da GPU seguem o mesmo ramo e não há divergência. Um valor fora de 1 a 5 cai no último ramo (`else`), que é a mistura, e o pixel nunca fica sem cor. O vertex shader não tem condições.

Não há laços em nenhum dos dois shaders. As três ondas do padrão procedural são uma soma de tamanho fixo, escrita termo a termo, e nenhum cálculo depende de vizinhos ou de iterações. Por isso não existe critério de parada, e cada pixel executa sempre o mesmo número de operações (Tabela 2.2).

5.7 Amostragem da textura

A textura é lida com `texture(baseTexture, uv)` nos modos 1, 2 e 4. `baseTexture` é um `sampler2D`, ou seja, uma imagem 2D já ligada ao seu sampler, que define como ela é lida. O sampler é criado na aplicação (`createTextureSampler`, Capítulo 4) com:

- **filtro** `LINEAR` na ampliação e na redução, que mistura os quatro texels mais próximos (com anisotropia, a GPU pode combinar mais amostras) e evita que o deslocamento lento do modo 2 "pule" de um texel para o próximo;
- **endereçamento** `REPEAT` em u e v, que aplica a Equação (3.7): quando o UV deslocado passa de 1, a leitura volta para o começo da textura;
- **mipmap** em modo `LINEAR`, mas com uma imagem de um só nível (`mipLevels = 1`). O nível de detalhe calculado pela GPU (Equação (3.13)) não tem efeito, e o modelo pequeno na tela pode apresentar serrilhado. Essa limitação está na Tabela 4.6;
- **anisotropia** ligada apenas se a GPU tiver suporte, no valor máximo que ela permite.

A imagem está no formato `R8G8B8A8_SRGB`, então a leitura já devolve a cor convertida para o espaço linear, onde a filtragem e a iluminação são calculadas. O UV deslocado é passado direto para `texture`, sem `fract`: a repetição fica a cargo do sampler. Isso mantém as derivadas do UV contínuas na emenda da repetição, de modo que a escolha de nível e a filtragem não criam uma linha visível ali. O padrão procedural e a grade não leem textura: são calculados diretamente a partir do UV.

5.8 Tratamento de limites e valores inválidos

- **UV fora de** $[0, 1]$**:** o modo 2 faz u crescer sem limite. Quem trata isso é o sampler (`REPEAT`), e não o shader. A grade de depuração usa `fract` para que u e v virem uma cor válida.
- **Normal nula ou não unitária:** a interpolação pode encurtar a normal, por isso ela é renormalizada com `normalize`. Uma normal nula daria um resultado inválido. Nos vértices, isso não acontece: o carregador do modelo (`ModelLoader.cpp`) calcula as normais ausentes somando as normais das faces vizinhas e troca as de comprimento quase zero ($\leq 10^{-6}$) por $(0, 1, 0)$. Dentro de um triângulo, a normal interpolada só chegaria perto de zero se os vértices tivessem normais quase opostas, o que não ocorre numa malha suave como a do modelo.
- **Modelo sem UV ou sem textura:** o carregador também trata esses casos antes do shader. Sem o atributo `TEXCOORD_0`, o UV de todos os vértices vira $(0, 0)$ e um aviso é mostrado; sem textura, é gerado um xadrez. O shader roda igual nos dois casos.

- **Luz por trás da superfície:** `max(dot, 0.0)` impede iluminação negativa, e o termo ambiente de 0,35 evita regiões totalmente pretas.
- **Energia procedural:** a soma das três ondas vai de -3 a 3 e é normalizada para $[0, 1]$ antes de servir de peso em `mix` e `smoothstep`.
- **Cor acima de 1:** o brilho do padrão procedural soma até 0,25 no verde e 0,35 no azul, e pode levar a cor acima de 1. O shader não aplica `clamp`: a aplicação escolhe para o color attachment o formato `B8G8R8A8_SRGB`, que é normalizado, e nesse tipo de formato a GPU limita cada canal a $[0, 1]$ ao gravar o pixel. Se a tela não oferecer esse formato, a aplicação usa o primeiro da lista, e o limite continua valendo se ele também for normalizado (`UNORM` ou `SRGB`).
- **Modo inválido:** qualquer valor fora de 1 a 5 cai no último ramo (mistura). Na aplicação, o modo começa em 4 e só muda pelas teclas 1 a 5.
- **Tempo muito grande:** `time` é um `float` de 32 bits. A partir de cerca de 18 horas de execução ($t \geq 2^{16}$ s), o menor passo representável de `time` chega a 0,008 s, quase meio quadro a 60 Hz, e o movimento passa a andar em degraus. O UV deslocado ($u + 0{,}10\,t$ e $u + 0{,}40\,t$) cresce junto com t e também perde precisão. Esse caso não é tratado no shader; a correção possível, na aplicação, está na Seção 2.5.

5.9 Resultado esperado

O vertex shader produz a posição de cada vértice na tela, e o fragment shader produz a cor de cada pixel do modelo. A Figura 5.2 mostra os cinco modos. Como o modo 2 é animado, a sequência dele ao longo do tempo está na Figura 7.6.

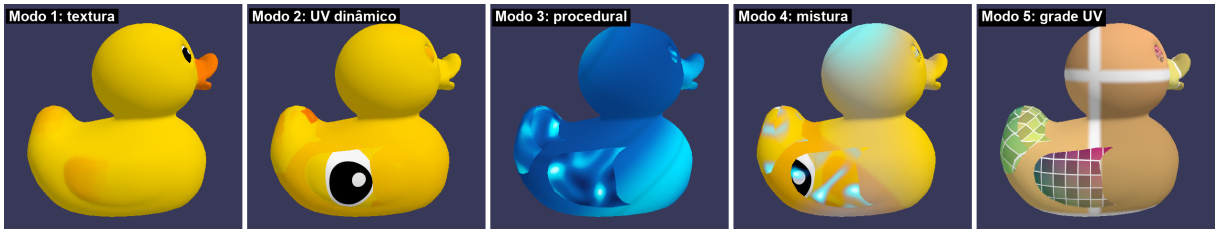


Figura 5.2: Resultado dos shaders na implementação, nos cinco modos (capturas próprias, `código/evidencias/`).

Resultado esperado de cada modo:

1. a textura original do modelo, parada;
2. a mesma textura deslizando sobre o modelo (uma volta completa a cada 10 segundos) e ondulando, sem que a malha se deforme. Na Figura 7.6, o olho passa da cabeça para o corpo;
3. ondas azuis e ciano geradas sem imagem, com uma grade sutil;
4. a textura deslizando, com as ondas mais fortes do padrão passando por cima;

5. a grade de depuração, que revela o esticamento e as costuras do UV.

5.10 Relação entre o pseudocódigo e a técnica

Heckbert descreve o texture mapping como a composição de dois mapeamentos: a parametrização da superfície, do espaço de textura para o objeto, e as transformações que levam o objeto à tela (1, p. 2). No pseudocódigo, a segunda parte é o passo 1 do vertex shader (Equação (3.2)). A primeira é o UV que vem do arquivo e é repassado no passo 2. O UV mapping **dinâmico** consiste em mudar essa parametrização a cada quadro sem mexer na malha, e é isso que `dynamicUV` faz: a geometria e a câmera continuam iguais, mas cada pixel passa a ler outro ponto da textura.

O fragment shader segue a varredura em ordem de tela, que Heckbert aponta como a mais comum: para cada pixel, encontra-se a sua pré-imagem no espaço de textura e filtra-se essa região (1, p. 3). No pseudocódigo, a posição na textura é dada pelo UV interpolado de cada fragmento, e a leitura é feita com `texture`. Heckbert separa ainda o mapeamento geométrico da filtragem (1, p. 1): aqui, o mapeamento é o cálculo do UV, e a filtragem é delegada ao sampler (`LINEAR` e mipmap).

O mapeamento **procedural** corresponde a `proceduralPattern`, que cria a cor "usando modelos matemáticos", na definição de Dong et al. (2, p. 1), sem ler imagem nenhuma. O padrão é uma soma de senoides, ideia que Heckbert registra nas séries de Fourier usadas como função de textura (1, p. 10). O modo 4 junta as duas abordagens: a textura com UV dinâmico, com o padrão procedural por cima. O modo 5 não faz parte do efeito: serve para enxergar a própria parametrização.

6 Aplicações práticas

Este capítulo apresenta produtos comerciais e ferramentas em uso na indústria que implementam UV mapping dinâmico ou mapeamento procedural, indicando para cada um o recurso específico, a equação correspondente no Capítulo 3 e a fonte que comprova o uso.

6.1 Motores de jogo: UV dinâmico como recurso de material

6.1.1 Unreal Engine: nó *Panner*

No editor de materiais da Unreal Engine, o nó *Panner* gera coordenadas UV que mudam de acordo com a entrada de tempo (9). Ele recebe as coordenadas originais e uma entrada de tempo, e tem duas propriedades de velocidade, *Speed X* e *Speed Y*, uma para cada direção. É exatamente a Equação (3.5) ($u' = u + s_u t$, $v' = v + s_v t$), empacotada como um bloco visual para artistas. A documentação oficial apresenta o nó como a forma de criar texturas "em deslocamento, ou em movimento" (9). O Rocket League é desenvolvido sobre a Unreal Engine 3 (10).

6.1.2 Unity: nó *Tiling And Offset* do Shader Graph

No Shader Graph da Unity, o nó *Tiling And Offset* aplica à coordenada de entrada a operação `Out = UV * Tiling + Offset` (11), que combina a escala (Equação (3.8)) e a translação (Equação (3.5)). Segundo a documentação, o nó é "comumente usado para mapas de detalhe e para rolar texturas ao longo do tempo" (11): basta ligar a saída de um nó de tempo à entrada *Offset*.

6.2 Ferramentas de autoria de texturas procedurais

6.2.1 Adobe Substance 3D Designer

O Substance Designer é um editor de texturas procedurais baseado em grafos de nós. Smelik et al. o citam entre os editores visuais por nós de uso disseminado em sistemas comerciais (3, p. 14). O uso em produção está documentado em uma entrevista com Eric Majka, um dos artistas líderes da Psyonix: todos os mapas recentes do Rocket League foram texturizados inteiramente no Designer, com *trim textures* (texturas em faixas reaproveitadas por várias peças) divididas em quartos para facilitar o UV mapping feito pelos modeladores. Os padrões da banda de rodagem dos pneus e os mapas de altura dos

aros também são gerados nele (10). A ferramenta é procedural na autoria: o artista edita parâmetros e o grafo gera as imagens finais, que o jogo usa como texturas comuns.

6.2.2 Blender: nós *Mapping*, *Noise Texture* e *Voronoi Texture*

O Blender, software livre de modelagem e renderização, oferece os dois lados da técnica como nós de material. O nó *Mapping* "transforma o vetor de entrada aplicando translação, rotação e escala" (12), a mesma transformação combinada da Equação (3.10). Com um valor animado na translação, ele produz UV dinâmico. Os nós *Noise Texture* e *Voronoi Texture* avaliam, respectivamente, um ruído de Perlin fractal e um ruído de Worley "nas coordenadas de textura de entrada" (13, 14). Na classificação de Dong et al., o primeiro é um ruído de gradiente em grade (método não estruturado, domínio espacial) e o segundo uma textura celular (método estruturado) (2, pp. 10, 16).

6.3 Modelagem procedural de mundos virtuais

Smelik et al. dedicam uma seção do levantamento aos sistemas disponíveis comercialmente, para mostrar "como os resultados de pesquisa já estão sendo aplicados na indústria" (3, p. 10). Os três seguintes estão entre os citados.

6.3.1 SpeedTree

O SpeedTree é um *middleware* para modelagem e renderização de grandes quantidades de vegetação procedural, com "ampla aplicação nas indústrias de jogos e de cinema" (3, p. 11). O fabricante o descreve como o conjunto de ferramentas de vegetação padrão da indústria, que combina geradores procedurais com edição manual (15). Segundo Smelik et al., o próprio motor do SpeedTree anima a vegetação e gerencia o nível de detalhe na renderização (3, p. 11).

6.3.2 Esri ArcGIS CityEngine

O CityEngine é um gerador de cidades baseado em gramáticas CGA (3, p. 12). Na documentação oficial, a modelagem é "descrita de forma abstrata, em um arquivo de regras", que refina iterativamente as formas até gerar os edifícios (16). O produto faz parte da plataforma ArcGIS, de geoprocessamento, e é voltado à geração de cidades e quarteirões inteiros em 3D. Uma regra CGA típica divide a fachada em andares e janelas que se repetem, o que, na textura, corresponde ao ladrilhamento por escala de UV da Equação (3.8).

6.3.3 SideFX Houdini

O Houdini permite que designers criem geradores procedurais a partir de pequenos blocos de construção (3, p. 12). Os autores ressaltam, porém, que o uso eficaz exige conhecimento avançado de como projetar esses procedimentos, um exemplo concreto da limitação de controle discutida no Capítulo 2.

6.4 Resumo

Tabela 6.1: Produtos analisados e sua relação com a técnica.

Produto	**Recurso**	**Técnica**	**Fonte**
Unreal Engine	Nó *Panner*	UV dinâmico, Eq. (3.5)	(9)
Unity	Nó *Tiling And Offset*	UV dinâmico, Eqs. (3.5) e (3.8)	(11)
Substance 3D Designer	Grafo de textura procedural	Procedural na autoria (Rocket League)	(10), (3, p. 14)
Blender	*Mapping*, *Noise*, *Voronoi*	UV dinâmico e procedural, Eq. (3.10)	(12, 13, 14)
SpeedTree	Vegetação procedural	Modelagem procedural e LOD	(15), (3, p. 11)
CityEngine	Regras CGA	Modelagem procedural, fachadas por repetição	(16), (3, p. 12)
Houdini	Geradores por nós	Modelagem procedural	(3, p. 12)

Os produtos se dividem em dois grupos. Nos motores de jogo (Unreal, Unity), a técnica é executada em tempo real, a cada quadro, no shader. Nas ferramentas de autoria (Substance, SpeedTree, CityEngine, Houdini), o procedural é usado durante a produção e o resultado é gravado como texturas e malhas comuns. Essa divisão segue o argumento de custo do Capítulo 2: o que é barato o suficiente roda no shader, e o que é caro ou difícil de controlar é gerado antes.

7 Análise em jogos

Os dois jogos foram jogados pelo grupo em um PlayStation 5. As capturas foram extraídas das transmissões ao vivo feitas por um integrante do grupo no canal *Nerd Geek TV*. Para cada jogo, a ficha traz os itens pedidos na análise: plataforma, situação de gameplay, momento, elemento visual, efeito, benefício e fonte técnica. Depois vêm as capturas marcadas e o vídeo.

Uma ressalva vale para os dois casos. Um jogo comercial não expõe seus shaders, e uma captura mostra só o resultado final. Quando o texto afirma como algo foi produzido, a afirmação vem de uma fonte do estúdio. Quando descreve um efeito observado na tela, ele é apresentado como *compatível* com a técnica, e não como prova da implementação.

Como ler as capturas. Os retângulos amarelos numerados marcam onde o UV mapping está sendo aplicado. A legenda de cada figura explica cada número: qual técnica aparece e em qual equação do Capítulo 3 ela se encaixa.

7.1 Rocket League

Tabela 7.1: Análise do Rocket League.

Jogo	*Rocket League* (Psyonix), motor Unreal Engine 3 (10)
Plataforma	PlayStation 5, executando a versão de PS4 por retrocompatibilidade (17), em 4K (*checkerboard*) a 60 quadros por segundo com HDR (18)
Situação analisada	(a) Partida online e o *replay* automático exibido após um gol; (b) tela inicial, com o carro do jogador em destaque
Momento	(a) Logo após o gol que deixou o placar em 4 a 3, faltando 3:18; (b) antes de entrar na partida
Elementos visuais	Carroceria, rodas e antena dos carros; bola; piso, faixas e painéis do estádio; rastro do *boost*
Efeito produzido	Pinturas e decalques que seguem a forma de cada carro; piso e faixas repetidos em todo o campo; rastro animado atrás do carro ao acelerar
Benefício	O mesmo layout UV de cada carro recebe dezenas de pinturas, decalques e rodas, a base da personalização do jogo, sem novos modelos. As *trim textures* reaproveitadas texturizam arenas inteiras com poucas imagens. O rastro animado dá retorno visual da aceleração sem geometria extra
Fonte técnica	Entrevista de um dos artistas líderes da Psyonix sobre o uso do Substance Designer no jogo (10)

Segundo a entrevista (10):

- os mapas recentes foram texturizados inteiramente no Substance Designer, com *trim textures* divididas em quartos para facilitar o UV mapping feito pelos modeladores;
- a banda de rodagem dos pneus e o relevo dos aros são gerados proceduralmente no Designer;
- um grafo combina mapas de curvatura e espessura com um mapa gerado a partir do layout UV de cada carro para criar os efeitos de borda dos decalques, e é reaplicado a cada carro novo;
- máscaras permitem aplicar a cor de cada time.

O rastro do *boost* é o efeito em que se espera UV dinâmico: uma faixa animada que acompanha o carro. Em motores como a Unreal, esse tipo de efeito é montado com texturas em deslocamento, como as do nó *Panner* (9). Como o estúdio não detalha a implementação do rastro, e parte dele é feita de partículas (estrelas e fumaça, no item usado pelo jogador

na live), ele é tratado aqui apenas como efeito compatível com a técnica.

Cena da Figura 7.1. Logo após um gol em partida online, o jogo exibe o *replay* da jogada. A câmera acompanha a bola perto do gol, com o carro adversário (azul) e o do jogador (vermelho, marcado "Você") no fundo.

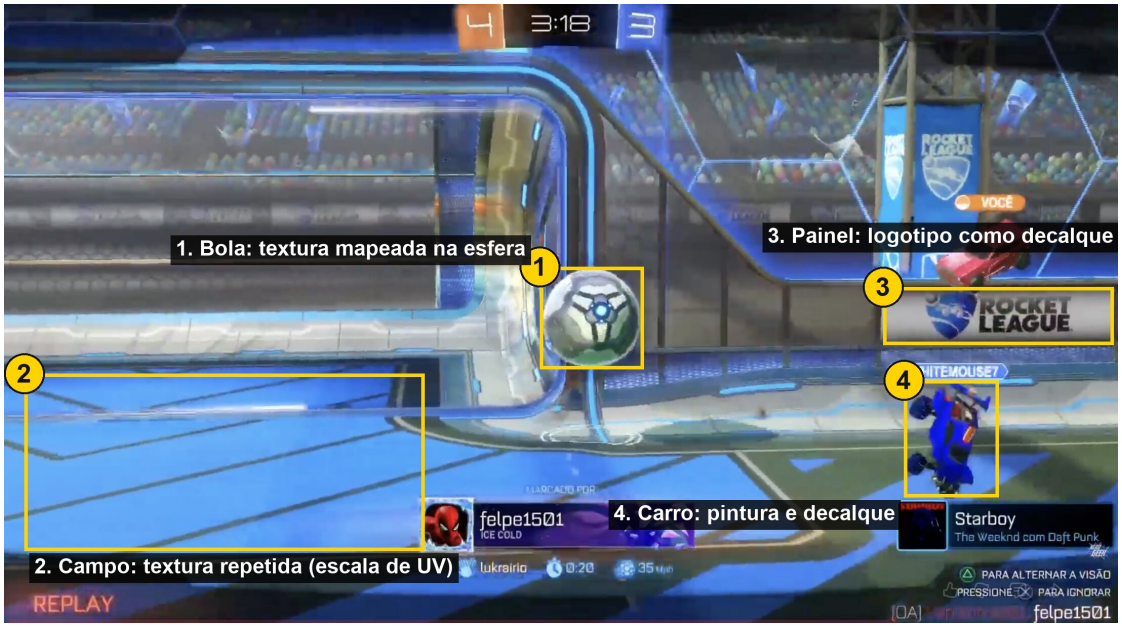


Figura 7.1: Rocket League (PS5), *replay* após um gol. (1) Bola: textura mapeada na esfera pelo UV. (2) Campo: o piso e as faixas repetem a mesma textura, ladrilhamento por escala de UV (Eq. 3.8). (3) Painel com logotipo: decalque aplicado sobre a região UV do painel. (4) Carro adversário: pintura sobre o UV do modelo.

Cena da Figura 7.2. Na tela inicial, antes da partida, o jogo exibe o carro do jogador parado no gramado, com a pintura e as rodas escolhidas na garagem.



Figura 7.2: Rocket League (PS5), tela inicial. (1) Carroceria: a pintura é uma textura sobre o layout UV do carro; os decalques, quando equipados, usam o mesmo layout. (2) Roda: banda de rodagem e aro gerados proceduralmente no Substance Designer (10). (3) Antena: esfera com textura própria. (4) Gramado: textura repetida (Eq. 3.8).

Vídeo. Transmissão ao vivo do grupo (PS5): https://www.youtube.com/live/eUX4mfkpFlQ. Nela aparecem a tela inicial com o carro, os *replays* após os gols e o rastro do *boost* durante as partidas.

7.2 Marvel's Spider-Man Remastered

Tabela 7.2: Análise do Marvel's Spider-Man Remastered.

Jogo	*Marvel's Spider-Man Remastered* (Insomniac Games)
Plataforma	PlayStation 5, versão nativa, lançada em 12 de novembro de 2020 (19)
Situação analisada	(a) Missão da história em que o Homem-Aranha observa um píer da polícia, em conversa por rádio com a "Técnica da Torre"; (b) exploração livre no alto de um arranha-céu de Manhattan; (c) menu de pausa, aba *Trajes*
Momento	(a) Durante a missão, com o personagem parado na borda do prédio; (b) no instante da notificação de desbloqueio do DLC "A Cidade que Nunca Dorme"; (c) na escolha do traje
Elementos visuais	Fachadas dos prédios; traje do personagem; veículos e piso do píer; água da baía e dos rios
Efeito produzido	Janelas repetidas ao longo dos andares; padrão de teia acompanhando a curvatura do corpo; decalques sobre os veículos e o píer; superfície de água com textura de ondas
Benefício	As fachadas repetem um módulo de janela, o que permite texturizar uma cidade inteira com poucas imagens. Cada traje é um conjunto de texturas sobre o UV do personagem, o que permite oferecer dezenas de trajes
Fonte técnica	Página da Insomniac sobre a versão remasterizada (modelos e materiais melhorados, reflexos com *ray tracing*) (20). Para a água, a Insomniac não publica a implementação; o uso de UV em deslocamento é justificado pela técnica documentada nos motores (9, 11) e demonstrado na implementação do grupo (Seção 7.3)

Neste jogo, a técnica aparece com mais clareza no UV tradicional: o padrão de teia dos trajes, os decalques dos veículos e as fachadas com janelas repetidas. A água da baía e dos rios é coberta por uma textura de ondas. Em jogos, superfícies de água desse tipo costumam ser animadas deslocando o UV da textura ao longo do tempo (Equações (3.5) e (3.6)), em vez de deformar a geometria, como no nó *Panner* (9). Como a Insomniac não publica a implementação e uma captura estática não registra movimento, a água é tratada aqui apenas como efeito compatível com a técnica. A demonstração do UV dinâmico em funcionamento é feita na implementação do grupo (Seção 7.3). Os reflexos sobre a água vêm de outra técnica, o *ray tracing* (20).

Cena da Figura 7.3. Durante uma missão, o Homem-Aranha está agachado na borda de um prédio, observando um píer com veículos da polícia, enquanto conversa por rádio

com a “Técnica da Torre”. A baía aparece ao fundo.

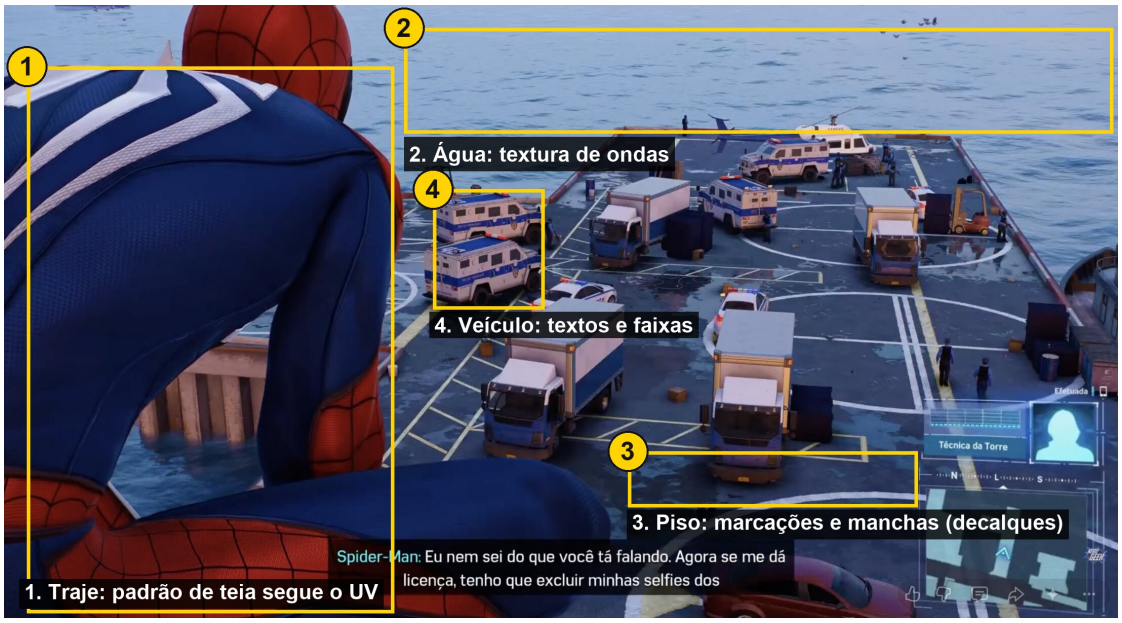


Figura 7.3: Marvel’s Spider-Man Remastered (PS5), missão no píer da polícia. (1) Traje: o padrão de teia e o relevo do tecido seguem o UV do personagem. (2) Água: superfície coberta por uma textura de ondas; em jogos, esse tipo de superfície costuma ser animado deslocando o UV (Eqs. 3.5 e 3.6), o que uma imagem estática não permite confirmar. (3) Piso do píer: marcações e manchas como decalques sobre uma textura repetida. (4) Veículos: textos e faixas mapeados pelo UV de cada modelo.

Cena da Figura 7.4. Na exploração livre, o personagem para no alto de um arranha-céu, com Manhattan e o rio à frente, no instante em que o jogo notifica o desbloqueio de um DLC.



Figura 7.4: Marvel's Spider-Man Remastered (PS5), exploração livre. (1) Fachada: o mesmo módulo de janelas se repete ao longo dos andares (Eq. 3.8). (2) Rio: superfície de água com textura de ondas, a mesma da Figura 7.3. (3) Traje do personagem, mapeado pelo UV.

Cena da Figura 7.5. No menu de pausa, aba *Trajes*, o jogo exibe o modelo do personagem com o traje selecionado ao lado da lista de trajes desbloqueados.

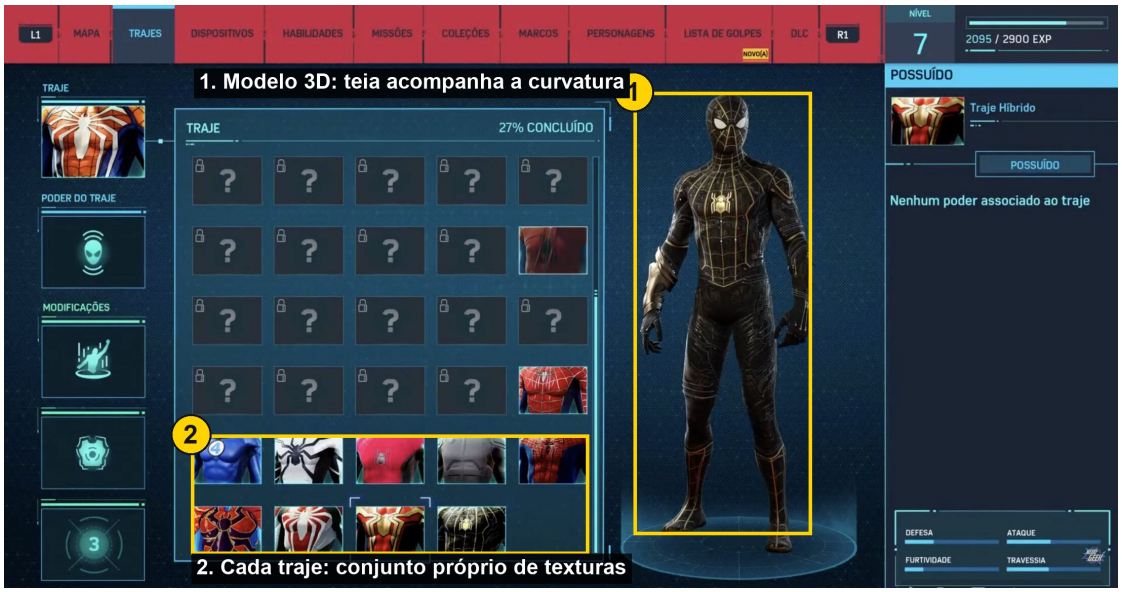


Figura 7.5: Marvel's Spider-Man Remastered (PS5), menu de trajes. (1) Modelo do personagem: as linhas da teia acompanham a curvatura do corpo, resultado da parametrização UV. (2) Lista de trajes: cada traje é um conjunto próprio de texturas.

Lives completas. Transmissões ao vivo do grupo (PS5): "Parte 5", https://www.youtube.com/live/mhFPk9TQWWA, e "Parte 3", https://www.youtube.com/live/RKo-mqZgxG8. Nelas aparecem as fachadas durante o balanço pela cidade, a baía e o menu de trajes.

7.3 Demonstração na implementação do grupo

Uma captura estática de um jogo comercial não registra movimento, e o código do jogo não está disponível. Por isso, a Figura 7.6 demonstra o UV dinâmico na implementação Vulkan do grupo, em que ele pode ser comprovado quadro a quadro. A rotação do modelo foi pausada (tecla Espaço). O primeiro quadro está no modo 1, com a textura original, e os outros três estão no modo 2, com cerca de dois segundos de intervalo. A câmera e a malha não mudam entre os quadros. O olho, marcado com um círculo, começa na cabeça e atravessa o corpo à medida que u cresce com $u' = u + 0{,}10\,t$ (Equação (3.5)).



Figura 7.6: Captura própria da implementação (Vulkan, `Duck.glb`): modo 1 e modo 2 em $t \approx 1$, 3 e 5 s, com a malha parada. O olho (círculo) desliza da cabeça para o corpo porque o UV é deslocado no fragment shader (função `dynamicUV`).

7.4 Comparação

Tabela 7.3: Técnicas observadas e onde aparecem nas capturas.

Técnica	**Equação**	**Onde aparece**
UV tradicional	(3.1), (3.4)	Fig. 7.1 (1, 4); Fig. 7.2 (1, 3); Fig. 7.3 (1, 4); Fig. 7.4 (3); Fig. 7.5 (1, 2)
Ladrilhamento por escala	(3.8), (3.7)	Fig. 7.1 (2); Fig. 7.2 (4); Fig. 7.4 (1)
Decalque	(3.1)	Fig. 7.1 (3); Fig. 7.3 (3)
Procedural (na produção)	—	Fig. 7.2 (2): pneu e aro gerados no Substance Designer (10)
UV dinâmico	(3.5), (3.6)	Fig. 7.6 (comprovado na implementação); Fig. 7.3 (2) e Fig. 7.4 (2) (água, compatível); rastro do *boost* no Rocket League (compatível)

Nos dois jogos, o UV tradicional é o que viabiliza a escala do conteúdo: dezenas de carros e trajes personalizáveis e uma cidade inteira texturizada com módulos repetidos. O UV dinâmico é esperado onde o jogo precisa de movimento contínuo e barato: a água no Spider-Man e o rastro do *boost* no Rocket League. Nos dois casos, porém, os estúdios

não publicam a implementação, e o rastro do *boost* ainda combina partículas (pequenas imagens que nascem e somem atrás do carro) com o brilho do rastro. Por isso, este trabalho trata esses efeitos como compatíveis com a técnica e demonstra o UV dinâmico em funcionamento na própria implementação (Figura 7.6), onde o deslocamento pode ser medido.

8 Resumo das publicações utilizadas

As versões de Heckbert e de Smelik et al. incluídas na pasta `docs/artigos/` são as versões dos autores (*preprints*), cuja paginação difere da revista. As páginas citadas referem-se a essas versões, para que possam ser conferidas nos arquivos entregues. Os mesmos resumos estão em `docs/resumos/`.

8.1 Heckbert (1986)

Título: *Survey of Texture Mapping.* **Autor:** Paul S. Heckbert. **Ano:** 1986 (IEEE Computer Graphics and Applications, v. 6, n. 11) (1).

Objetivo. Organizar o conhecimento sobre texture mapping disponível até meados dos anos 1980, separando o problema em mapeamento geométrico e filtragem (1, p. 1).

Principais conceitos.

- O mapeamento como composição de uma parametrização da superfície com a projeção para a tela (1, p. 2).
- Os parâmetros que podem ser mapeados (cor, normal, especular, transparência, deslocamento) (1, p. 2).
- Os mapeamentos afim, bilinear e projetivo, e a interpolação correta em perspectiva, com uma divisão por pixel (1, pp. 4–5).
- A taxonomia dos filtros (convolução direta, pré-filtragem e séries de Fourier) (1, p. 7), com uma tabela de custo e memória em que a pirâmide de mipmaps ocupa 4/3 da textura e a interpolação trilinear custa 8 acessos por pixel (1, pp. 9–10).

Relação com o tema. É a base do UV mapping: define o espaço (u, v), mostra como ele chega a cada pixel e quanto custa ler a textura corretamente. O UV dinâmico altera justamente a parametrização descrita pelo autor. O artigo é anterior às GPUs programáveis e não trata de UV animado.

Informações aproveitadas. Composição do mapeamento e ordem de varredura (Cap. 2); interpolação com correção de perspectiva e seleção de mipmap (Cap. 3); aliasing, custo da filtragem trilinear e memória dos mipmaps (Caps. 2 e 4).

8.2 Dong et al. (2020)

Título: *Survey of Procedural Methods for Two-Dimensional Texture Generation.* **Autores:** Junyu Dong, Jun Liu, Kang Yao, Mike Chantler, Lin Qi, Hui Yu e Muwei Jian. **Ano:**

2020 (Sensors, v. 20, n. 4, 1135) (2).

Objetivo. Revisar e comparar, pela aparência que produzem, os métodos de geração procedural de texturas bidimensionais estáticas. Texturas sólidas, jogos e animação ficam explicitamente fora do escopo (2, p. 2).

Principais conceitos.

- A definição de textura procedural como imagem criada por um modelo matemático (2, p. 1).
- A classificação em métodos estruturados (autômatos celulares, reação-difusão, textons, Voronoi, texturas celulares), não estruturados (funções de ruído no domínio da frequência e no espacial, como Gabor e Perlin) e de pós-processamento (2, pp. 3–4).
- Uma avaliação perceptual de 20 métodos, com 450 amostras e 40 participantes (2, pp. 19–20), resumida em uma tabela de aplicações típicas de cada método (2, p. 22).

Relação com o tema. Fundamenta o lado procedural da técnica: o que é uma textura gerada por função, quando ela é vantajosa (tamanho ilimitado, sem costuras, famílias de variações) e quais são seus problemas (controle de parâmetros, aliasing, dificuldade de reproduzir texturas reais) (2, pp. 2, 13, 23).

Informações aproveitadas. Definição e taxonomia do procedural, critérios de quando usar e limitações (Cap. 2); classificação dos nós procedurais do Blender (Cap. 6); a ressalva de que o tempo relatado (menos de um segundo por imagem, em computador de mesa) mede a geração de uma imagem inteira, e não a avaliação por fragmento em tempo real (2, p. 21).

8.3 Smelik et al. (2014)

Título: *A Survey on Procedural Modelling for Virtual Worlds.* **Autores:** Ruben M. Smelik, Tim Tutenel, Rafael Bidarra e Bedrich Benes. **Ano:** 2014 (Computer Graphics Forum, v. 33, n. 6) (3).

Objetivo. Revisar os métodos de modelagem procedural para mundos virtuais e entender por que eles ainda são pouco adotados por profissionais criativos (3, p. 1).

Principais conceitos.

- A amplificação de dados: poucos parâmetros geram grande variedade de conteúdo, inclusive sob demanda na GPU (3, p. 1).
- A organização dos métodos por tipo de conteúdo (terreno, vegetação, água, estradas, cidades, edifícios, interiores) e a avaliação de cada um por facilidade de uso, controle e família de técnica (3, p. 2).

- Uma seção sobre sistemas comerciais (Terragen, SpeedTree, CityEngine, Houdini) (3, pp. 10–12).
- Os dois obstáculos à adoção: a falta de controle intuitivo e a falta de integração entre métodos (3, pp. 12–15).

Relação com o tema. Mostra o procedural em escala de mundo e o problema de controle que também afeta as texturas procedurais. O levantamento exclui a geração de texturas (3, p. 2), por isso é usado para o contexto de produção e para os produtos reais, e não para a texturização em si.

Informações aproveitadas. Amplificação de dados e custo da produção manual (quando usar), baixa controlabilidade e desempenho interativo (limitações e custo) (Cap. 2); SpeedTree, CityEngine, Houdini e Substance Designer como produtos reais (Cap. 6).

8.4 Relação entre as publicações

As três publicações cobrem camadas diferentes do problema. Heckbert trata de como uma coordenada (u, v) chega a um pixel e quanto custa ler a textura. Dong et al. tratam do conteúdo procedural da textura em si. Smelik et al. tratam do procedural em escala de mundo e do problema de controle que limita seu uso. Nenhuma delas trata diretamente do UV dinâmico em tempo real. Esse ponto é coberto pela implementação do grupo e pelas fontes técnicas dos motores e estúdios (Capítulos 4, 6 e 7).

9 Conclusão

O UV mapping associa cada ponto de uma superfície 3D a um ponto de uma textura 2D. Seguindo Heckbert (1, p. 2), o processo foi tratado como a composição de uma parametrização da superfície com a projeção para a tela. O UV dinâmico altera essa parametrização a cada quadro sem tocar na malha, e o mapeamento procedural substitui a imagem por uma função matemática (2, p. 1).

As três abordagens resolvem problemas diferentes. O UV tradicional é a escolha para detalhes únicos e posicionais, como rostos, logotipos e decalques. O UV dinâmico é a escolha para movimento repetitivo (água, lava, esteiras, campos de energia), com o custo de uma amostragem e poucas operações por fragmento. O procedural é a escolha para grandes áreas sem repetição e para famílias de variações (2, p. 2). Em troca, ele exige mais cálculo por fragmento, não tem mipmaps que o protejam do aliasing e é difícil de controlar. Dong et al. descrevem o ajuste de parâmetros como um processo demorado (2, p. 23), e Smelik et al. apontam a baixa controlabilidade como o principal motivo de os métodos procedurais ainda não substituírem a modelagem manual (3, p. 1).

A análise das equações mostrou que a pergunta mais importante para o desempenho é onde cada cálculo é executado. Transformações lineares podem ir para a CPU ou para o vertex shader. Deformações não lineares e funções procedurais precisam ficar no fragment shader, que executa centenas de vezes mais que o vertex shader no modelo usado. No Vulkan, a técnica depende de um sampler com endereçamento `REPEAT`, de um descriptor set que exponha a textura e os parâmetros, e de um mecanismo para enviar o tempo a cada quadro. A implementação usa um Uniform Buffer porque os command buffers são pré-gravados; push constants seriam a alternativa natural se fossem regravados por quadro.

Os produtos analisados mostram as duas formas de uso na indústria. Nos motores de jogo, o UV dinâmico é um recurso de material executado em tempo real, como o *Panner* da Unreal e o *Tiling And Offset* da Unity. Nas ferramentas de autoria, como Substance Designer, SpeedTree e CityEngine, o procedural é usado durante a produção. No Rocket League, as texturas das arenas e das rodas são criadas com o Substance Designer durante a produção, e o jogo as aplica como texturas comuns sobre o UV de cada modelo (10). No Marvel's Spider-Man Remastered, a técnica aparece nos trajes, nos decalques e nas fachadas repetidas, e a água da baía é um caso compatível com UV dinâmico. Como os estúdios não publicam a implementação desses efeitos, o UV dinâmico em funcionamento foi demonstrado na implementação do grupo, em que o deslocamento da textura pode ser medido quadro a quadro.

A implementação entregue tem limitações conhecidas, discutidas ao longo do texto: a textura não tem mipmaps, o tempo não é mantido periódico, a translação de UV ainda

é feita por pixel e a grade procedural não tem antialiasing analítico. Esses pontos são o caminho natural para uma próxima versão, junto com a divisão dos parâmetros entre Uniform Buffer e push constants.

Referências

1 HECKBERT, Paul S. Survey of Texture Mapping. **IEEE Computer Graphics and Applications**, v. 6, n. 11, p. 56–67, 1986. Páginas citadas referem-se à versão do autor em artigos/. DOI: 10.1109/MCG.1986.276672.

2 DONG, Junyu *et al.* Survey of Procedural Methods for Two-Dimensional Texture Generation. **Sensors**, v. 20, n. 4, p. 1135, 2020. DOI: 10.3390/s20041135.

3 SMELIK, Ruben M. *et al.* A Survey on Procedural Modelling for Virtual Worlds. **Computer Graphics Forum**, v. 33, n. 6, p. 31–50, 2014. Páginas citadas referem-se à versão do autor em artigos/. DOI: 10.1111/cgf.12276.

4 KHRONOS GROUP. **Vulkan Tutorial: Image view and sampler**. Disponível em: https://docs.vulkan.org/tutorial/latest/06_Texture_mapping/01_Image_view_and_sampler.html. Acesso em: 30 out. 2026.

5 KHRONOS GROUP. **vkCmdPushConstants(3)**. Disponível em: https://docs.vulkan.org/refpages/latest/refpages/source/vkCmdPushConstants.html. Acesso em: 30 out. 2026.

6 KHRONOS GROUP. **Vulkan Guide: Mapping Data to Shaders**. Disponível em: https://docs.vulkan.org/guide/latest/mapping_data_to_shaders.html. Acesso em: 30 out. 2026.

7 KHRONOS GROUP. **Vulkan Specification: Limits**. Disponível em: https://docs.vulkan.org/spec/latest/chapters/limits.html. Acesso em: 30 out. 2026.

8 KHRONOS GROUP. **Vulkan Tutorial: Generating Mipmaps**. Disponível em: https://docs.vulkan.org/tutorial/latest/09_Generating_Mipmaps.html. Acesso em: 30 out. 2026.

9 EPIC GAMES. **Coordinates Material Expressions in Unreal Engine**. Disponível em: https://dev.epicgames.com/documentation/en-us/unreal-engine/coordinates-material-expressions-in-unreal-engine. Acesso em: 30 out. 2026.

10 ADOBE SUBSTANCE 3D. **Between Modeling and Texturing: Substance Designer in Rocket League**. 13 fev. 2019. Disponível em: https://magazine.substance3d.com/between-modeling-and-texturing-substance-designer-in-rocket-league/. Acesso em: 30 out. 2026.

11 UNITY TECHNOLOGIES. **Tiling And Offset Node**. Disponível em: https://docs.unity3d.com/Packages/com.unity.shadergraph@14.0/manual/Tiling-And-Offset-Node.html. Acesso em: 30 out. 2026.

12 BLENDER FOUNDATION. **Mapping Node**. Disponível em: https://docs.blender.org/manual/en/latest/render/shader_nodes/utilities/vector/mapping.html. Acesso em: 30 out. 2026.

13 BLENDER FOUNDATION. **Noise Texture Node**. Disponível em: https://docs.blender.org/manual/en/latest/render/shader_nodes/textures/noise.html. Acesso em: 30 out. 2026.

14 BLENDER FOUNDATION. **Voronoi Texture Node**. Disponível em: https://docs.blender.org/manual/en/latest/render/shader_nodes/textures/voronoi.html. Acesso em: 30 out. 2026.

15 UNITY TECHNOLOGIES. **SpeedTree**. Disponível em: https://unity.com/products/speedtree. Acesso em: 30 out. 2026.

16 ESRI. **Introduction to CityEngine**. Disponível em: https://doc.arcgis.com/en/cityengine/latest/get-started/get-started-about-cityengine.htm. Acesso em: 30 out. 2026.

17 SONY INTERACTIVE ENTERTAINMENT. **Rocket League (PlayStation Store)**. Disponível em: https://store.playstation.com/en-us/product/UP2002-CUSA01163_00-ROCKETLEAGUENA01. Acesso em: 30 out. 2026.

18 PSYONIX. **Jogue Rocket League no Xbox Series X, Series S e no PlayStation 5**. 10 nov. 2020. Disponível em: https://www.rocketleague.com/news/play-rocket-league-on-xbox-series-x--series-s--and-playstation-5. Acesso em: 30 out. 2026.

19 SONY INTERACTIVE ENTERTAINMENT. **Marvel's Spider-Man Remastered**. Disponível em: https://www.playstation.com/en-us/games/marvels-spider-man-remastered/. Acesso em: 30 out. 2026.

20 INSOMNIAC GAMES. **What is different in Marvel's Spider-Man Remastered from the original game?** Disponível em: https://support.insomniac.games/hc/en-us/articles/46670426851475-What-is-different-in-Marvel-s-Spider-Man-Remastered-from-the-original-game. Acesso em: 30 out. 2026.

A Checklist dos requisitos

Revisão bibliográfica

- ☒ Definição, funcionamento, problema resolvido, vantagens, limitações, custo e quando usar (Capítulo 2).
- ☒ Três publicações científicas citadas ao longo do texto, com um resumo de cada (Capítulo 8 e `docs/resumos/`).
- ☒ Equações com variáveis, finalidade, momento de uso e relação com a renderização (Capítulo 3).
- ☒ Relação com o Vulkan: shaders, estágios, buffers, texturas, attachments, fluxo de dados e otimizações (Capítulo 4).
- ☒ Sete aplicações reais, com onde a técnica é usada e o benefício (Capítulo 6).
- ☒ Dois jogos, com plataforma, situação, momento, elemento, efeito, benefício e fonte técnica (Capítulo 7).
- ☒ Capturas próprias marcadas, com legenda e texto da cena (Capítulo 7).
- ☒ Links das lives de gameplay do grupo no PDF e em `docs/videos.txt`; demonstração do UV dinâmico em funcionamento na implementação (Figura 7.6).

Arquitetura do pipeline Vulkan

- ☒ Diagrama com legenda de cores e setas, e arquivo-fonte `docs/diagrama/pipeline_vulkan.dot` (Figura 4.1).
- ☒ Fluxo de dados, com entradas e saídas de cada estágio (Tabela 4.2).
- ☒ Render Pass, Subpass e attachments; descriptor sets e bindings; layout e finalidade dos buffers (Capítulo 4).
- ☒ Barreiras de memória e dependência do subpass com todos os campos, transições de layout e dependências entre CPU e GPU (Tabelas 4.4 e 4.5).
- ☒ Justificativa das decisões arquiteturais (Capítulo 4).

Shaders

- ☒ Pseudocódigo em GLSL, comentado trecho a trecho, em `docs/pseudocodigo/uv_mapping.vert` e `uv_mapping.frag`, reproduzido por completo no Capítulo 5.
- ☒ Objetivo, fluxo (Figura 5.1), entradas e saídas, equações e variáveis (Tabela 5.2), condições e laços, amostragem, tratamento de limites, resultado esperado e relação com a técnica (Capítulo 5).

Código

☒ Aplicação Vulkan em C++17, dividida em arquivos por assunto em `código/src/`, cobrindo: instância, validação, superfície, GPU e filas, dispositivo lógico, swapchain, render pass, pipeline, framebuffers, comandos, sincronização, loop principal e liberação dos recursos.

☒ Shaders do Capítulo 5 compilados para SPIR-V pelo CMake (`glslc`), carregados como shader modules e ligados aos estágios do pipeline (`código/README.md`, seção 6).

☒ Documento com compilação, execução, bibliotecas e versões, arquivos principais e funcionalidades prontas e futuras (`código/README.md`), script `build_windows.bat` e evidências da execução (`código/evidencias/`).